

Unit 5: Concurrency, Backup and Recovery

Concurrency

- In a single-user database, a user can modify data without concern for other users modifying the same data at the same time. However, in a multiuser database, statements within multiple simultaneous transactions may update the same data. Transactions executing simultaneously must produce meaningful and consistent results.
- A multiuser database must provide the following:
 - ☞ The assurance that users can access data at the same time (*data concurrency*)
 - ☞ The assurance that each user sees a consistent view of the data (*data consistency*), including visible changes made by the user's own transactions and committed transactions of other users
- To describe consistent transaction behavior when transactions run concurrently, database researchers have defined a transaction isolation model called **serializability**. A serializable transaction operates in an environment that makes it appear as if no other users were modifying data in the database.

Transaction Isolation Levels

- Transaction isolation enables each active transaction to operate as if there were no other transactions active in the system.
- Isolation levels determine if row-level locks are acquired when performing read operations. When a statement is issued to update a table, locks are acquired to prevent other transactions from modifying the same data until the updating transaction commits or rolls back and then releases its locks.
- The Isolation connection attribute sets the isolation level for a connection. The isolation level cannot be changed in the middle of a transaction.

Transaction isolation levels:

- **ANSI Read Committed isolation:** The read committed isolation level is the recommended mode of operation for most applications, and is the default mode. It enables transactions that are reading data to run concurrently with a transaction that is updating the same data. TimesTen makes multiple versions of data items to allow non-serializable read and write operations to proceed in parallel.
 - Read operations do not block write operations and write operations do not block read operations, even when they read and write the same data. Read operations do not acquire locks on scanned rows. Write operations acquire locks that are held until the transaction commits or rolls back. Readers share a committed copy of the data, whereas a writer has its own uncommitted version. Therefore, when a transaction reads an item that is being updated by another in-progress transaction, it sees the committed version of that item. It cannot see an uncommitted version of an in-progress transaction.
 - Read committed isolation level provides for better concurrency at the expense of decreased isolation because of the possibility of non-repeatable reads or phantom rows within a transaction. If an application runs the same query multiple times within the same transaction, the commit of an update from another transaction may cause the results from the second read operation to retrieve different results. A phantom row can appear (in modified form) in two different read operations within the same transaction, as the result of an early release of read locks during the transaction.

To set read committed isolation level, if previously modified since this is the default, do one of the following:

- ☞ Connect with the Isolation connection attribute set to 1. You can also modify this value with the ALTER SESSION SQL statement.
`ALTER SESSION SET ISOLATION_LEVEL=READ COMMITTED;`
- ☞ When using ttIsql, run isolation 1 or isolation READ_COMMITTED.
- ☞ ODBC applications can run the SQLSetConnectOption ODBC function with the SQL_TXN_ISOLATION flag set to SQL_TXN_READ_COMMITTED.

☞ JDBC applications can run the *setTransactionIsolation* JDBC method of the Connection object to TRANSACTION_READ_COMMITTED.

— **ANSI Serializable isolation:** All locks acquired within a transaction by a read or write operation are held until the transaction commits or rolls back. Read operations block write operations, and write operations block read operations. But read operations do not block other read operations.

- If an initial transaction reads a row, a second transaction can also read the row.
- A row that has been read by an initial transaction cannot be updated or deleted by a second transaction until the initial transaction commits or rolls back. However, the second transaction can insert rows that are not in the same range that are locked by the first transaction. For example, if the first transaction is reading rows 1 through 10, then the second transaction cannot update rows 1 through 10. However, the second transaction can insert into rows greater than 10.
- A row that has been inserted, updated or deleted by an initial transaction cannot be accessed in any way by a second transaction until the initial transaction either commits or rolls back.
- Serializable isolation level provides for repeatable reads and increased isolation at the expense of decreased concurrency. A transaction that runs the same query multiple times within the same transaction is guaranteed to see the same result set each time. Other transactions cannot update or delete any of the returned rows, nor can they insert a new row that satisfies the query predicate.
- To set the isolation level to serializable isolation, do one of the following:
 - Connect with the Isolation connection attribute set to 0. You can also modify this value with the ALTER SESSION SQL statement.

```
ALTER SESSION SET ISOLATION_LEVEL= SERIALIZABLE;
```

- When using *ttIsql*, run *isolation 0* or *isolation SERIALIZABLE*.

- ODBC applications can run the *SQLSetConnectOption* ODBC function with the `SQL_TXN_ISOLATION` flag set to `SQL_TXN_SERIALIZABLE`.
 - JDBC applications can run the ***setTransactionIsolation*** JDBC method of the Connection object to `TRANSACTION_SERIALIZABLE`.
- To ensure that materialized views are always in a consistent state, all view maintenance operations are performed under Serializable isolation, even when the transaction is in read committed isolation. This means that the transaction obtains read locks for any data items read during view maintenance. However, the transaction releases the read locks at the end of the `INSERT`, `UPDATE` or `CREATE VIEW` statement that triggered the view maintenance, instead of holding them until the end of the transaction.

Levels of Locking in Oracle (Locking Granularities)

Oracle supports row-level locks, table-level locks and database-level locks.

- **Row-level locking:** Transactions usually obtain locks on the individual rows that they access. Row-level locking is the recommended mode of operation because it provides the finest granularity of concurrency control. It allows concurrent transactions to update different rows of the same table. However, row-level locking requires space in the database's temporary memory region to store lock information. Row-level locking is the default. However, if it has been modified to another type of locking and you want to re-enable row-level locking, do one of the following:
 - Set the *LockLevel* connection attribute to 0.
 - In *TimesTen* Classic, call the *ttLockLevel* built-in procedure with the *lockLevel* parameter set to *Row*. This procedure changes the lock level between row-level and database-level locking on the next transaction and for all subsequent transactions for this connection.
 - Run the *ttOptSetFlag* built-in procedure to set the *RowLock* parameter to 1, which enables the optimizer to consider using row locks.

- **Table-level locking:** Table-level locking is recommended when concurrent transactions access different tables or a transaction accesses most of the rows of a particular table. Table-level locking provides better concurrency than database-level locking. Row-level locking provides better concurrency than table-level locking. Table-level locking requires only a small amount of space in the temporary memory region to store lock information.

Table-level locking provides the best performance for the following:

- Queries that access a significant number of rows of a table
- When there are very few concurrent transactions that access a table
- When temporary space is inadequate to contain all row locks that an operation, such as a large insert or a large delete, might acquire

To enable table-level locking, run the *ttOptSetFlag* procedure to set the *TblLock* parameter to 1, which enables the optimizer to consider using table locks. In addition, set *RowLock* to 0 so that the optimizer does not consider row-level locks.

- **Database-level locking:** Database-level locking serializes all transactions, which effectively allows no concurrency on the database. When a transaction is started, it acquires an exclusive lock on the database, which ensures that there is no more than one active transaction in the database at any given time. It releases the lock when the transaction is completed.

Database-level locking can provide better performance than row-level locking, due to reduced locking overhead. In addition, it provides higher throughput than row-level locking when running a single stream of transactions such as a bulk load operation. However, its applicability is limited to applications that never run multiple concurrent transactions. With database-level locking, every transaction effectively runs in ANSI Serializable isolation, since concurrent transactions are disallowed.

To enable database-level locking, do one of the following:

- Set the *LockLevel* connection attribute to 1.
- In *TimesTen Classic*, call the *ttLockLevel* built-in procedure with the *lockLevel* parameter set to DS. This procedure changes the lock level between row-level and database-level locking on

the next transaction and for all subsequent transactions for this connection.

Setting Wait Time for Acquiring a Lock

- Set the LockWait connection attribute to the maximum amount of time that a statement waits to acquire a lock before it times out. The default is 10 seconds. If a statement within a transaction waits for a lock and the lock wait interval has elapsed, an error is returned. After receiving the error, the application can reissue the statement.
- Lock wait intervals are imprecise due to the scheduling of the database's managing subdaemon process to detect lock timeouts. This imprecision does not apply to zero-second timeouts, which are always immediately reported. The lock wait interval does not apply to blocking checkpoints.
- The database's managing subdaemon process checks every two seconds to see if there is a deadlock in the database among concurrent transactions. If a deadlock occurs, an error is returned to one of the transactions involved in the deadlock cycle. The transaction that receives the error must rollback in order to allow the other transactions involved in the deadlock to proceed.
- When running a workload of high lock-contention potential, consider setting the LockWait connection attribute to a smaller value for faster return of control to the application, or setting LockWait to a larger value to increase the successful lock grant ratio (with a risk of decreased throughput).

Overview of the Oracle Database Locking Mechanism

- A **lock** is a mechanism that prevents destructive interactions. Interactions are destructive when they incorrectly update data or incorrectly alter underlying data structures, between transactions accessing shared data. Locks play a crucial role in maintaining database concurrency and consistency.
- The database maintains several different types of locks, depending on the operation that acquired the lock.
- In general, the database uses two types of locks: **exclusive locks** and **share locks**. Only one exclusive lock can be obtained on a resource such as a row or a table, but many share locks can be obtained on a single resource.

- Locks affect the interaction of readers and writers. A reader is a query of a resource, whereas a writer is a statement modifying a resource. The following rules summarize the locking behavior of Oracle Database for readers and writers:
 - ☞ A row is locked only when modified by a writer.
When a statement updates one row, the transaction acquires a lock for this row only. By locking table data at the row level, the database minimizes contention for the same data. Under normal circumstances the database does not escalate a row lock to the block or table level.
 - ☞ A writer of a row blocks a concurrent writer of the same row.
If one transaction is modifying a row, then a row lock prevents a different transaction from modifying the same row simultaneously.
 - ☞ A reader never blocks a writer.
Because a reader of a row does not lock it, a writer can modify this row. The only exception is a `SELECT ... FOR UPDATE` statement, which is a special type of `SELECT` statement that does lock the row that it is reading.
 - ☞ A writer never blocks a reader.
When a row is being changed by a writer, the database uses undo data to provide readers with a consistent view of the row.

Lock conversion and Lock escalation

- The automatic conversion of a table lock of lower restrictiveness to one of higher restrictiveness. For example, suppose a transaction issues a `SELECT ... FOR UPDATE` for an employee and later updates the locked row. In this case, the database automatically converts the row share table lock to a row exclusive table lock. Oracle Database performs lock conversion as necessary.
- A situation that occurs in some databases when numerous locks are held at one level of granularity (for example, rows) and the database raises the locks to a higher level of granularity (for example, table). Oracle Database never escalates locks. Lock escalation greatly increases the probability of deadlocks.

Lock Duration

- Oracle Database automatically releases a lock when some event occurs so that the transaction no longer requires the resource.
- Usually, the database holds locks acquired by statements within a transaction for the duration of the transaction. These locks prevent destructive interference such as dirty reads, lost updates, and destructive DDL from concurrent transactions.
- Oracle Database releases all locks acquired by the statements within a transaction when it commits or rolls back. Oracle Database also releases locks acquired after a savepoint when rolling back to the savepoint. However, only transactions not waiting for the previously locked resources can acquire locks on the now available resources. Waiting transactions continue to wait until after the original transaction commits or rolls back completely.

Methods for acquiring locks in Oracle

There are two main methods for acquiring locks in Oracle:

1. Automatic Locking (Default)
2. Explicit Locking (Manual)

Overview of Automatic Locks

- Oracle Database automatically locks a resource on behalf of a transaction to prevent other transactions from doing something that requires exclusive access to the same resource.
- The database automatically acquires different types of locks at different levels of restrictiveness depending on the resource and the operation being performed.
- This is the most common and recommended approach for most situations. It happens transparently behind the scenes whenever a transaction modifies data (INSERT, UPDATE, DELETE). Oracle automatically acquires locks on the data to ensure data consistency by preventing other transactions from modifying the same data concurrently.
- Oracle Database locks are divided into the following categories shown in the

Lock	Description
DML Locks	Protect data. For example, table locks lock entire tables, while row locks lock selected rows.
DDL Locks	Protect the structure of schema objects—for example, the dictionary definitions of tables and views.
System Locks	Protect internal database structures such as data files. Latches, mutexes, and internal locks are

DML Locks

- A DML lock, also called a **data lock**, guarantees the integrity of data accessed concurrently by multiple users.
- For example, a DML lock prevents two customers from buying the last copy of a book available from an online bookseller. DML locks prevent destructive interference of simultaneous conflicting DML or DDL operations.
- DML statements automatically acquire the following types of locks:
 - Row Locks (TX)
 - Table Locks (TM)

Row Locks (TX)

- A row lock, also called a TX lock, is a lock on a single row of table. A transaction acquires a row lock for each row modified by an INSERT, UPDATE, DELETE, MERGE, or SELECT ... FOR UPDATE statement. The row lock exists until the transaction commits or rolls back.
- Row locks primarily serve as a queuing mechanism to prevent two transactions from modifying the same row. The database always locks a modified row in exclusive mode so that other transactions cannot modify the row until the transaction holding the lock commits or rolls back. Row locking provides the finest grain locking possible and so provides the best possible concurrency and throughput.
- If a transaction obtains a lock for a row, then the transaction also acquires a lock for the table containing the row. The table lock prevents conflicting DDL operations that would override data changes in a current transaction. Following illustrates an

update of the third row in a table. Oracle Database automatically places an exclusive lock on the updated row and a subexclusive lock on the table.

EMPLOYEE_ID	LAST_NAME	EMAIL	HIRE_DATE	JOB_ID	MANAGER_ID	DEPARTMENT_ID
 100	King	SKING	17-JUN-87	AD_PRES		90
101	Kochhar	NKOCHHAR	21-SEP-89	AD_VP	100	90
102	De Hann	LDEHANN	13-JAN-93	AD_VP	100	90
103	Hunold	AHUNOLD	03-JAN-90	IT_PROG	102	60

 Table lock acquired
 Exclusive row lock (TX) acquired
 Row being updated

Figure 1: Row and Table Locks

Table Locks (TM)

- A table lock, also called a TM lock, is acquired by a transaction when a table is modified by an INSERT, UPDATE, DELETE, MERGE, SELECT with the FOR UPDATE clause, or LOCK TABLE statement.
- DML operations require table locks to reserve DML access to the table on behalf of a transaction and to prevent DDL operations that would conflict with the transaction.
- A table lock can be held in any of the following modes:

- *Row Share (RS)*

This lock, also called a subshare table lock (SS), indicates that the transaction holding the lock on the table has locked rows in the table and intends to update them. A row share lock is the least restrictive mode of table lock, offering the highest degree of concurrency for a table.

- *Row Exclusive Table Lock (RX)*

This lock, also called a subexclusive table lock (SX), generally indicates that the transaction holding the lock has updated table rows or issued SELECT ... FOR UPDATE. An SX lock allows other transactions to query, insert, update,

delete, or lock rows concurrently in the same table. Therefore, SX locks allow multiple transactions to obtain simultaneous SX and subshare table locks for the same table.

- *Share Table Lock (S)*

A share table lock held by a transaction allows other transactions to query the table (without using SELECT ... FOR UPDATE), but updates are allowed only if a single transaction holds the share table lock. Because multiple transactions may hold a share table lock concurrently, holding this lock is not sufficient to ensure that a transaction can modify the table.

- *Share Row Exclusive Table Lock (SRX)*

This lock, also called a share-subexclusive table lock (SSX), is more restrictive than a share table lock. Only one transaction at a time can acquire an SSX lock on a given table. An SSX lock held by a transaction allows other transactions to query the table (except for SELECT ... FOR UPDATE) but not to update the table.

- *Exclusive Table Lock (X)*

This lock is the most restrictive, prohibiting other transactions from performing any type of DML statement or placing any type of lock on the table.

DDL Locks

- A data dictionary (DDL) lock protects the definition of a schema object while an ongoing DDL operation acts on or refers to the object.
- Only individual schema objects that are modified or referenced are locked during DDL operations. The database never locks the whole data dictionary.
- Oracle Database acquires a DDL lock automatically on behalf of any DDL transaction requiring it. Users cannot explicitly request DDL locks. For example, if a user creates a stored procedure, then Oracle Database automatically acquires DDL locks for all schema objects referenced in the procedure definition. The DDL locks prevent these objects from being altered or dropped before procedure compilation is complete

Exclusive DDL Locks

- An exclusive DDL lock prevents other sessions from obtaining a DDL or DML lock.
- Most DDL operations require exclusive DDL locks for a resource to prevent destructive interference with other DDL operations that might modify or reference the same schema object. For example, DROP TABLE is not allowed to drop a table while ALTER TABLE is adding a column to it, and vice versa.
- Exclusive DDL locks last for the duration of DDL statement execution and automatic commit. During the acquisition of an exclusive DDL lock, if another DDL lock is held on the schema object by another operation, then the acquisition waits until the older DDL lock is released and then proceed

Share DDL Locks

- A share DDL lock for a resource prevents destructive interference with conflicting DDL operations, but allows data concurrency for similar DDL operations.
- For example, when a CREATE PROCEDURE statement is run, the containing transaction acquires share DDL locks for all referenced tables. Other transactions can concurrently create procedures that reference the same tables and acquire concurrent share DDL locks on the same tables, but no transaction can acquire an exclusive DDL lock on any referenced table.
- A share DDL lock lasts for the duration of DDL statement execution and automatic commit. Thus, a transaction holding a share DDL lock is guaranteed that the definition of the referenced schema object remains constant during the transaction.

System Locks

Oracle Database uses various types of system locks to protect internal database and memory structures. These mechanisms are inaccessible to users because users have no control over their occurrence or duration.

Latches

- A latch is a simple, low-level serialization mechanism that coordinates multiuser access to shared data structures, objects, and files.
- Latches protect shared memory resources from corruption when accessed by multiple processes. Specifically, latches protect data structures from the following situations:
 - Concurrent modification by multiple sessions
 - Being read by one session while being modified by another session
 - Deallocation (aging out) of memory while being accessed

Mutexes

- A mutual exclusion object (mutex) is a low-level mechanism that prevents an object in memory from aging out or from being corrupted when accessed by concurrent processes. A mutex is similar to a latch, but whereas a latch typically protects a group of objects, a mutex protects a single object.
- Mutexes provide several benefits:
 - A mutex can reduce the possibility of contention.
Because a latch protects multiple objects, it can become a bottleneck when processes attempt to access any of these objects concurrently. By serializing access to an individual object rather than a group, a mutex increases availability.
 - A mutex consumes less memory than a latch.
 - When in shared mode, a mutex permits concurrent reference by multiple sessions.

Internal Locks

Internal locks are higher-level, more complex mechanisms than latches and mutexes and serve various purposes. The database uses the following types of internal locks:

- *Dictionary cache locks*

These locks are of very short duration and are held on entries in dictionary caches while the entries are being modified or used. They guarantee that statements being parsed do not see inconsistent object definitions. Dictionary cache locks can be shared or exclusive. Shared locks are released when the parse is complete, whereas exclusive locks are released when the DDL operation is complete.

- *File and log management locks*

These locks protect various files. For example, an internal lock protects the control file so that only one process at a time can change it. Another lock coordinates the use and archiving of the online redo log files. Data files are locked to ensure that multiple instances mount a database in shared mode or that one instance mounts it in exclusive mode. Because file and log locks indicate the status of files, these locks are necessarily held for a long time.

- *Tablespace and undo segment locks*

These locks protect tablespaces and undo segments. For example, all instances accessing a database must agree on whether a tablespace is online or offline. Undo segments are locked so that only one database instance can write to a segment.

Overview of Manual Data Locks

- You can manually override the Oracle Database default locking mechanisms.
- Oracle Database performs locking automatically to ensure data concurrency, data integrity, and statement-level read consistency. However, overriding the default locking is useful in situations such as the following:
 - Applications require transaction-level read consistency or repeatable reads. In this case, queries must produce consistent data for the duration of the transaction, not reflecting changes by other transactions. You can achieve transaction-level read consistency by using explicit locking, read-only transactions, serializable transactions, or by overriding default locking.

- Applications require that a transaction have exclusive access to a resource so that the transaction does not have to wait for other transactions to complete.
- You can override Oracle Database automatic locking at the session or transaction level. At the session level, a session can set the required transaction isolation level with the ALTER SESSION statement. At the transaction level, transactions that include the following SQL statements override Oracle Database default locking:
 - The SET TRANSACTION ISOLATION LEVEL statement
 - The LOCK TABLE statement (which locks either a table or, when used with views, the base tables)
 - The SELECT ... FOR UPDATE statement
- Locks acquired by the preceding statements are released after the transaction ends or a rollback to savepoint releases them.
- If Oracle Database default locking is overridden at any level, then the database administrator or application developer should ensure that the overriding locking procedures operate correctly. The locking procedures must satisfy the following criteria: data integrity is guaranteed, data concurrency is acceptable, and deadlocks are not possible or are appropriately handled.

Overview of User-Defined Locks

- With Oracle Database Lock Management services, you can define your own locks for a specific application.
- For example, you might create a lock to serialize access to a message log on the file system. Because a reserved user lock is the same as an Oracle Database lock, it has all the Oracle Database lock functionality including deadlock detection. User locks never conflict with Oracle Database locks, because they are identified with the prefix UL.
- The Oracle Database Lock Management services are available through procedures in the DBMS_LOCK package. You can include statements in PL/SQL blocks that:
 - Request a lock of a specific type
 - Give the lock a unique name recognizable in another procedure in the same or in another instance
 - Change the lock type

— Release the lock

Possible Causes of Contention

- In Oracle databases, contention refers to situations where multiple processes or transactions compete for the same resources, leading to potential performance degradation or blocking. Some common causes of contention in Oracle databases include:
 - **Lock contention:** When multiple transactions or sessions attempt to access the same data concurrently and one transaction holds a lock on the resource, other transactions may need to wait, leading to contention. This can happen with row-level locks, table-level locks, or even at higher levels of granularity.
 - **Latch contention:** Latches are low-level serialization mechanisms used to protect in-memory structures. If multiple processes attempt to acquire the same latch simultaneously, contention may occur, leading to performance issues. Common latch contention includes library cache latch, shared pool latch, and buffer cache latch contention.
 - **Resource contention:** Resources such as CPU, memory, and disk I/O are finite and shared among different processes. If multiple processes compete for these resources, contention may arise, leading to performance degradation. For example, excessive CPU consumption by one process can lead to contention for CPU resources among other processes.
 - **I/O contention:** Disk I/O operations, especially in high-demand environments, can become a bottleneck. If multiple processes concurrently request disk reads or writes, contention may occur, leading to increased response times and decreased throughput.
 - **Hotspots:** Hotspots refer to specific database objects, such as tables or indexes, that experience disproportionately high levels of activity compared to other objects. Access to these hotspots can become a point of contention, especially if they are heavily accessed by multiple transactions simultaneously.
 - **Contention for shared resources:** Oracle databases often utilize shared resources such as the buffer cache, library cache, and shared pool. If multiple processes concurrently access or modify these shared resources, contention may occur, impacting overall system performance.

- **Concurrency control mechanisms:** Oracle employs various concurrency control mechanisms such as read and write locks, multiversion concurrency control (MVCC), and serialization to ensure data consistency in multi-user environments. However, these mechanisms can also introduce contention, especially in scenarios with high transaction concurrency.
- Addressing contention issues often involves careful monitoring, performance tuning, and, in some cases, redesigning applications or database structures to reduce contention points. Additionally, techniques such as partitioning, indexing, and caching can be employed to alleviate contention in specific scenarios.

Detect Lock Contention

Detecting lock contention in Oracle involves monitoring database performance metrics and analyzing database wait events to identify instances where transactions are waiting for locks. Here are some methods and tools commonly used to detect lock contention in Oracle:

- **Database Performance Monitoring:** Utilize Oracle Enterprise Manager (OEM) or third-party monitoring tools to track database performance metrics such as CPU usage, I/O throughput, and memory utilization. Sudden spikes or prolonged periods of high resource consumption may indicate potential contention issues.
- **AWR Reports:** Oracle's Automatic Workload Repository (AWR) collects and maintains performance statistics for the database. Generating and analyzing AWR reports can provide insights into wait events, including lock-related waits such as "enqueue" and "buffer busy waits."
- **V\$ Views:** Query dynamic performance views such as V\$SESSION, V\$LOCK, and V\$LOCKED_OBJECT to identify sessions holding locks, sessions waiting for locks, and the objects being locked. Monitoring these views can help pinpoint specific lock contention scenarios.
- **Database Tracing:** Enable SQL trace or session tracing for specific sessions experiencing performance issues related to lock contention. Trace files can reveal detailed information about SQL statements, wait events, and lock acquisition attempts, aiding in diagnosis and troubleshooting.
- **Oracle Trace File Analyzer (TFA):** TFA is a diagnostic tool provided by Oracle to collect and analyze diagnostic data, including trace files, alert logs, and other diagnostic

information. Analyzing TFA output can help identify lock contention issues and their root causes.

- **Application Profiling:** Examine application code and SQL queries to identify inefficient locking patterns, such as holding locks for extended periods or acquiring unnecessary locks. Optimizing application logic and transaction management can help reduce lock contention.
- **Real-Time Monitoring Tools:** Use real-time monitoring tools such as Oracle Trace Data Viewer (OTDV) or Oracle Performance Analyzer (OPA) to monitor database activity, including lock-related events, in real-time. These tools provide visibility into current lock contention issues and can aid in proactive performance management.
- **Custom Scripts and Alerts:** Develop custom scripts or configure database alerts to monitor key performance indicators related to lock contention, such as the number of blocked sessions, the duration of lock waits, and the frequency of lock-related wait events. Set up alerts to notify administrators of potential contention issues.

Here are some additional tips for detecting lock contention:

- *Identify Lock Wait Times:* Analyze the average and peak wait times for lock-related events. High or increasing wait times suggest potential contention.
- *Correlate Lock Waits with Transactions:* Look for correlations between lock waits and specific transactions or SQL statements. This can help identify problematic queries or application logic.
- *Monitor Lock Mode:* Analyze the lock mode causing contention (e.g., exclusive lock, share lock). This can help determine the severity and potential solutions.

By leveraging these methods and tools, database administrators can effectively detect lock contention in Oracle databases, diagnose underlying causes, and implement appropriate measures to mitigate performance impact and ensure optimal database performance.

Preventing Locking Problems in Oracle

Preventing locking problems in Oracle databases involves implementing best practices for database design, application development, and system configuration. Here are some key strategies to prevent locking problems:

- **Optimize SQL Queries:** Write efficient SQL queries that access data in a consistent and predictable manner. Use appropriate indexes, query hints, and optimizer hints to minimize unnecessary locks and reduce contention.
- **Transaction Management:** Design transactions to be as short and focused as possible to minimize the duration of locks. Avoid long-running transactions that hold locks for extended periods, as they can increase the risk of contention and blocking.
- **Use Explicit Locking:** When necessary, use explicit locking mechanisms such as `SELECT ... FOR UPDATE` or `SELECT ... FOR SHARE` to control access to data and avoid unnecessary contention. Be cautious when using exclusive locks, as they can cause blocking and impact concurrency.
- **Concurrency Control:** Implement appropriate concurrency control mechanisms such as optimistic locking or multiversion concurrency control (MVCC) to allow concurrent access to data while ensuring consistency and avoiding blocking.
- **Avoid Table-level Locks:** Minimize the use of table-level locks, as they can cause unnecessary contention and blocking. Instead, use row-level locks or finer-grained locking mechanisms to reduce the scope of locking.
- **Indexing Strategies:** Use indexes strategically to improve query performance and reduce the need for full table scans and locks. Evaluate index usage and consider creating or modifying indexes to optimize access paths and minimize contention.
- **Partitioning:** Partition large tables to distribute data across multiple segments, reducing contention on individual partitions and improving parallelism and scalability. Partitioning can help mitigate locking problems, especially in high-volume transactional environments.
- **Monitor Locking Activity:** Regularly monitor database performance metrics, wait events, and locking activity using tools such as Oracle Enterprise Manager (OEM), AWR reports, and dynamic performance views. Identify and address any instances of excessive locking or contention proactively.
- **Database Configuration:** Configure Oracle database parameters such as `INITRANS`, `MAXTRANS`, and `PCTFREE` appropriately to optimize locking behavior and reduce contention. Adjust these parameters based on workload characteristics and concurrency requirements.

- **Application Design:** Design applications with concurrency and scalability in mind, using techniques such as connection pooling, session management, and distributed transactions to minimize contention and maximize throughput.

By following these practices and adopting a proactive approach to database design, application development, and system configuration, you can effectively prevent locking problems in Oracle databases and ensure optimal performance, concurrency, and reliability.

Resolve Conflicts

- In databases like Oracle, conflicts can arise in two main scenarios:
 - *Data Modification Conflicts:* This occurs when multiple transactions (processes trying to modify data) attempt to update the same data concurrently (at the same time). This can lead to inconsistencies in the data if not handled properly.
 - *Replication Conflicts:* This happens when data is replicated across multiple Oracle databases, and changes are made on different sites. These conflicts arise when updates try to modify the same data on different replicas simultaneously.
- The approach to resolving conflicts depends on the scenario:
 - *Data Modification:* Techniques like optimistic locking or custom logic using triggers or stored procedures can help identify and handle conflicts gracefully.
(Examples: Optimistic Locking, Conflict Resolution Logic, Isolation Levels, etc.)
 - *Replication:* Utilize reports to identify conflicts, choose the resolution approach (automatic or manual), and review conflicting changes to ensure data integrity across replicas. (Examples: Merge Replication, Conflict Resolution Strategies, etc)
- Resolving conflicts in Oracle typically involves addressing issues related to data consistency, concurrency, and contention that may arise in multi-user environments. Here are several approaches to resolving conflicts in Oracle:
 - **Isolation Levels:** Adjust the isolation level of transactions to control the visibility of changes made by concurrent transactions. Oracle supports different isolation levels such as Read Committed, Serializable, and Read-Only, which offer varying levels of consistency and concurrency control.
 - **Locking Mechanisms:** Use appropriate locking mechanisms to prevent conflicting access to shared resources. Oracle provides various types of locks, including row-level

locks, table-level locks, and application-level locks, which can be used to control access and enforce data integrity.

- **Optimistic Locking:** Implement optimistic locking techniques to detect and prevent conflicts at the application level. This involves adding a version number or timestamp to records and checking for modifications before committing changes. If conflicts are detected, appropriate actions such as retrying or displaying a conflict resolution dialog can be taken.
- **Conflict Resolution Strategies:** Define conflict resolution strategies to handle conflicting updates gracefully. Depending on the application requirements, conflicts can be resolved automatically by applying predefined rules (e.g., last writer wins), or they can be resolved manually by notifying users and allowing them to resolve conflicts based on context.
- **Merge Replication:** If conflicts occur in distributed environments with replicated data, use merge replication to synchronize changes and resolve conflicts automatically. Merge replication allows multiple copies of a dataset to be modified independently and later reconciled to ensure consistency across replicas.
- **Data Partitioning:** Partition large datasets to reduce contention and minimize the likelihood of conflicts. By partitioning data based on usage patterns, geographical location, or other criteria, you can distribute workload and isolate transactions, reducing the risk of conflicts.
- **Application Design:** Design applications with conflict resolution in mind, incorporating features such as versioning, auditing, and logging to track changes and detect conflicts. Use transaction management techniques to ensure atomicity, consistency, isolation, and durability (ACID properties) of transactions.
- **Monitoring and Analysis:** Regularly monitor database performance, concurrency, and contention using tools such as Oracle Enterprise Manager (OEM), AWR reports, and dynamic performance views. Analyze wait events, locking activity, and transaction statistics to identify potential conflicts and performance bottlenecks proactively.
- **Training and Documentation:** Provide training and documentation to users and developers on best practices for conflict resolution and concurrency control. Educate users on how to recognize and resolve conflicts effectively to minimize data inconsistency and improve overall system reliability.

By implementing these approaches and strategies, you can effectively resolve conflicts in Oracle databases and maintain data consistency, concurrency, and reliability in multi-user environments.

Undo Management

- Oracle Database creates and manages information that is used to roll back, or undo, changes to the database. Such information consists of records of the actions of transactions, primarily before they are committed. These records are collectively referred to as undo.
- Undo records are used to:
 - Roll back transactions when a ROLLBACK statement is issued
 - Recover the database
 - Provide read consistency
 - Analyze data as of an earlier point in time by using Oracle Flashback Query
 - Recover from logical corruptions using Oracle Flashback features
- When a ROLLBACK statement is issued, undo records are used to undo changes that were made to the database by the uncommitted transaction. During database recovery, undo records are used to undo any uncommitted changes applied from the redo log to the data files. Undo records provide read consistency by maintaining the before image of the data for users who are accessing the data at the same time that another user is changing it.

Undo Data Overview

When a transaction modifies the database, Oracle copies the original data before modifying it. The original copy of the modified data is called undo data. Saving this information is necessary for the following reasons:

- To undo any uncommitted changes made to the database in the event that a rollback operation is necessary. A rollback operation can be the result of a user who wants to undo the changes of a misguided or unintentional transaction, or it can be part of a recovery operation.
- To provide read consistency, which means that each user can get a consistent view of data, even while other uncommitted changes may be occurring against the data.

For example, if a user issues a query at 10:00 a.m. and the query lasts for 15 minutes, then the query results should reflect the entire state of the data at 10:00 a.m., regardless of updates or inserts by other users during the query.

- To enable certain Flashback features, namely Flashback Query and Flashback Table, which enable you to view or recover data to a previous point in time.

In Oracle databases, undo data plays a crucial role in maintaining data consistency, ensuring transactional integrity, and supporting various database features such as rollback, read consistency, and flashback operations.

Undo data, also known as rollback data, records the changes made to data by transactions in the database. It maintains a historical record of the before-image of data blocks, allowing transactions to be rolled back or undone if necessary.

Monitoring and administering undo

- Monitoring and administering undo in Oracle is essential for maintaining database performance, managing transactions, and ensuring data consistency.

Monitoring Undo

- **Automatic Workload Repository (AWR) Reports** provide insights into undo usage, including undo segments, undo space consumption, and undo blocks written. Look for metrics like "Undo Blocks Written per Transaction" or "Average Undo Blocks per Transaction" to identify potential undo bottlenecks.
- **V\$UNDOSTAT** dynamic view offers real-time information about undo segments, their size, and usage statistics. You can use SQL queries to analyze undo segment usage patterns and identify potential space constraints.

Example

```
SELECT segment_name,
       used_blocks,
       MAX(used_blocks) high_water_mark,
       MAX(used_blocks) - used_blocks reclaimable_blocks
```

```
FROM V$UNDOSTAT
```

```
GROUP BY segment_name;
```

This query retrieves undo segment names, used blocks, high-water mark (maximum used blocks), and reclaimable blocks (unused space) for analysis.

- **Analyze wait events** like free undo buffer waits or buffer busy waits (undo) to identify situations where transactions might be waiting for undo resources.

Administering Undo:

- **Managing Undo Retention:** Oracle automatically manages undo segments based on a high-water mark approach. However, you can adjust the retention period using parameters like UNDO_RETENTION or by setting retention guidelines within Automatic Storage Management (ASM).
- **Resizing Undo Tablespaces:** If undo space consumption becomes excessive, you can resize the undo tablespace to accommodate increased needs. Use the ALTER TABLESPACE statement to modify the tablespace size.
- **Monitoring Undo Segments:** Regularly review undo segment usage and consider dropping inactive segments to reclaim space. Use the DBMS_UNDO package functions like DROP_SEGMENT to manage inactive segments.
- **Tuning for Performance:** In high-concurrency environments, consider optimizing undo settings like the number of undo segments or the undo block size to improve performance and reduce undo-related wait times.

Overview of Automatic Undo Management

Oracle provides a fully automated mechanism, referred to as automatic undo management, for managing undo information and space. With automatic undo management, the database manages undo segments in an undo tablespace.

Automatic undo management is the default mode for a newly installed database

You can also create an undo tablespace explicitly.

When the database instance starts, the database automatically selects the first available undo tablespace. If no undo tablespace is available, then the instance starts without an undo tablespace and stores undo records in the SYSTEM tablespace

If the database contains multiple undo tablespaces, then you can optionally specify at startup that you want to use a specific undo tablespace. This is done by setting the UNDO_TABLESPACE initialization parameter, as shown in this example:

```
UNDO_TABLESPACE = undotbs_01
```

The following is a summary of the initialization parameters for undo management:

Initialization Parameter	Description
UNDO_MANAGEMENT	If AUTO or null, enables automatic undo management. If MANUAL, sets manual undo management mode. The default is AUTO.
UNDO_TABLESPACE	Optional, and valid only in automatic undo management mode. Specifies the name of an undo tablespace. Use only when the database has multiple undo tablespaces and you want to direct the database instance to use a particular undo tablespace.

When automatic undo management is enabled, if the initialization parameter file contains parameters relating to manual undo management, they are ignored.

The Undo Retention Period

- The undo retention period is the minimum amount of time that Oracle Database attempts to retain old undo information before overwriting it.
- After a transaction is committed, undo data is no longer needed for rollback or transaction recovery purposes. However, for consistent read purposes, long-running queries may require this old undo information for producing older images of data blocks. Furthermore, the success of several Oracle Flashback features can also depend upon the availability of older undo information. For these reasons, it is desirable to retain the old undo information for as long as possible.

- Old (committed) undo information that is older than the current undo retention period is said to be expired and its space is available to be overwritten by new transactions. Old undo information with an age that is less than the current undo retention period is said to be unexpired and is retained for consistent read and Oracle Flashback operations.
- Oracle Database automatically tunes the undo retention period based on undo tablespace size and system activity. You can optionally specify a minimum undo retention period (in seconds) by setting the UNDO_RETENTION initialization parameter. The exact impact this parameter on undo retention is as follows:
 - The UNDO_RETENTION parameter is ignored for a fixed size undo tablespace. The database always tunes the undo retention period for the best possible retention, based on system activity and undo tablespace size.
 - For an undo tablespace with the AUTOEXTEND option enabled, the database attempts to honor the minimum retention period specified by UNDO_RETENTION. When space is low, instead of overwriting unexpired undo information, the tablespace auto-extends. If the MAXSIZE clause is specified for an auto-extending undo tablespace, when the maximum size is reached, the database may begin to overwrite unexpired undo information.

NOTE: You can determine the current retention period by querying the TUNED_UNDORETENTION column of the V\$UNDOSTAT view.

Retention Guarantee

- To guarantee the success of long-running queries or Oracle Flashback operations, you can enable retention guarantee.
- If retention guarantee is enabled, then the specified minimum undo retention is guaranteed; the database never overwrites unexpired undo data even if it means that transactions fail due to lack of space in the undo tablespace. If retention guarantee is not enabled, then the database can overwrite unexpired undo when space is low, thus lowering the undo retention for the system. This option is disabled by default.
- You enable retention guarantee by specifying the RETENTION GUARANTEE clause for the undo tablespace when you create it with either the CREATE DATABASE or CREATE UNDO TABLESPACE statement. Or,

you can later specify this clause in an ALTER TABLESPACE statement. You disable retention guarantee with the RETENTION NOGUARANTEE clause.

- You can use the DBA_TABLESPACES view to determine the retention guarantee setting for the undo tablespace. A column named RETENTION contains a value of GUARANTEE, NOGUARANTEE, or NOT APPLY, where NOT APPLY is used for tablespaces other than the undo tablespace.

Configuring Undo Retention in Oracle

- Undo retention in Oracle determines how long the database retains undo data, allowing transactions to rollback changes. Here are the methods to configure undo retention:

1. *Using the UNDO_RETENTION Initialization Parameter:*

- This is the simplest method and sets a minimum retention period for all undo segments in seconds.
- You can modify this parameter dynamically using the ALTER SYSTEM statement:

```
ALTER SYSTEM SET UNDO_RETENTION = 1800;

-- Retains undo data for 30 minutes
```

2. *Automatic Undo Retention Management:*

- Oracle 19c offers automatic undo retention management based on the high-water mark approach.
- The database automatically tunes the undo retention period based on factors like:
 - Undo tablespace size
 - System workload and undo usage patterns
- This method is generally recommended as it provides dynamic adjustments based on actual needs.

3. *Setting Retention Guidelines within Automatic Storage Management (ASM):*

- If you're using ASM for undo tablespaces, you can define retention guidelines within ASM.

- This allows you to specify minimum and maximum retention periods for undo data.
- Use the ALTER DISKGROUP statement with the UNDO_RETENTION clause to configure these guidelines.

Managing Undo Tablespaces

- You manage undo tablespaces by completing tasks such as creating, altering, and dropping them. You can also switch undo tablespaces and establish user quotas for undo space.

Creating an Undo Tablespace

- Although Database Configuration Assistant (DBCA) automatically creates an undo tablespace for new installations, there may be occasions when you want to manually create an undo tablespace
- When you are creating a database, you can create an undo tablespace with the CREATE DATABASE statement. In an existing database, you can create an undo tablespace with the CREATE UNDO TABLESPACE statement.
- There are two methods of creating an undo tablespace. The first method creates the undo tablespace when the CREATE DATABASE statement is issued. This occurs when you are creating a new database, and the instance is started in automatic undo management mode (UNDO_MANAGEMENT = AUTO). The second method is used with an existing database. It uses the CREATE UNDO TABLESPACE statement.
- You cannot create database objects in an undo tablespace. It is reserved for system-managed undo data.
- The following statement illustrates using the UNDO TABLESPACE clause in a CREATE DATABASE statement. The undo tablespace is named undotbs_01 and one data file, /u01/oracle/rbdb1/undo0101.dbf, is allocated for it.

```
CREATE DATABASE rbdb1
    CONTROLFILE REUSE
```

```

    .
    .
```

.

```

    UNDO          TABLESPACE          undotbs_01          DATAFILE
    '/u01/oracle/rbdb1/undo0101.dbf';

```

— The CREATE UNDO TABLESPACE statement is the same as the CREATE TABLESPACE statement, but the UNDO keyword is specified. The database determines most of the attributes of the undo tablespace, but you can specify the DATAFILE clause.

— This example creates the undotbs_02 undo tablespace with the AUTOEXTEND option:

```

CREATE UNDO TABLESPACE undotbs_02
        DATAFILE '/u01/oracle/rbdb1/undo0201.dbf' SIZE 2M REUSE
        AUTOEXTEND ON;

```

— You can create multiple undo tablespaces, but only one of them can be active at any one time.

Altering an Undo Tablespace

- You can alter an undo tablespaces using the ALTER TABLESPACE statement.
- However, since most aspects of undo tablespaces are system managed, you need only be concerned with the following actions:
 - ☞ Adding a data file
 - ☞ Renaming a data file
 - ☞ Bringing a data file online or taking it offline
 - ☞ Beginning or ending an open backup on a data file
 - ☞ Enabling and disabling undo retention guarantee
- These are also the only attributes you are permitted to alter.
- If an undo tablespace runs out of space, or you want to prevent it from doing so, you can add more files to it or resize existing data files.
- The following example adds another data file to undo tablespace undotbs_01:

```

ALTER TABLESPACE undotbs_01
        ADD DATAFILE '/u01/oracle/rbdb1/undo0102.dbf' AUTOEXTEND ON NEXT 1M
        MAXSIZE UNLIMITED;

```

- You can use the ALTER DATABASE...DATAFILE statement to resize or extend a data file.

Dropping an Undo Tablespace

- Use the DROP TABLESPACE statement to drop an undo tablespace.
- The following example drops the undo tablespace undotbs_01:
`DROP TABLESPACE undotbs_01;`
- An undo tablespace can only be dropped if it is not currently used by any instance. If the undo tablespace contains any outstanding transactions (for example, a transaction died but has not yet been recovered), the DROP TABLESPACE statement fails. However, since DROP TABLESPACE drops an undo tablespace even if it contains unexpired undo information (within retention period), you must be careful not to drop an undo tablespace if undo information is needed by some existing queries.
- DROP TABLESPACE for undo tablespaces behaves like DROP TABLESPACE...INCLUDING CONTENTS. All contents of the undo tablespace are removed.

Switching Undo Tablespaces

- You can switch from using one undo tablespace to another. Because the UNDO_TABLESPACE initialization parameter is a dynamic parameter, the ALTER SYSTEM SET statement can be used to assign a new undo tablespace.
- The following statement switches to a new undo tablespace:
`ALTER SYSTEM SET UNDO_TABLESPACE = undotbs_02;`

Assuming undotbs_01 is the current undo tablespace, after this command successfully executes, the instance uses undotbs_02 in place of undotbs_01 as its undo tablespace.

- If any of the following conditions exist for the tablespace being switched to, an error is reported and no switching occurs:
 - The tablespace does not exist
 - The tablespace is not an undo tablespace

- The tablespace is already being used by another instance (in an Oracle RAC environment only)

Establishing User Quotas for Undo Space

- You can use the Oracle Database Resource Manager to establish user quotas for undo space. The Database Resource Manager directive UNDO_POOL allows DBAs to limit the amount of undo space consumed by a group of users (resource consumer group).
- You can specify an undo pool for each consumer group. An undo pool controls the amount of total undo that can be generated by a consumer group. When the total undo generated by a consumer group exceeds its undo limit, the current UPDATE transaction generating the undo is terminated. No other members of the consumer group can perform further updates until undo space is freed from the pool.
- When no UNDO_POOL directive is explicitly defined, users are allowed unlimited undo space.

Retention Period for Flashback Queries

- Flashback queries are a powerful feature in Oracle databases that allow you to view historical data as of a specific point in time. This enables you to:
 - Analyze past data states for troubleshooting or auditing purposes.
 - Recover data accidentally deleted or modified.
 - Compare data across different points in time.
- The ability to perform flashback queries in Oracle relies on undo data, which essentially stores information about changes made to the database. The retention period for this undo data determines how far back in time you can perform a flashback query.
- In Oracle, the retention period for Flashback Queries depends on several factors:
 - **Undo Retention:** Flashback Queries rely on undo data, which is used to reconstruct the state of the data at a particular point in the past. The retention period for undo data is determined by the UNDO_RETENTION parameter in the database initialization parameters. This parameter specifies the minimum amount of time undo data should be retained before it can be overwritten. If

Flashback Queries are enabled and undo retention is sufficient, you can query historical data using Flashback Query within the specified retention period.

- **Undo Tablespace Size:** The size of the undo tablespace also plays a role. If the undo tablespace runs out of space before the retention period expires, older undo data may be overwritten prematurely, limiting the duration for which Flashback Queries can be performed.
 - **Flashback Query Syntax:** Flashback Queries are executed using the AS OF clause in SQL statements. The timestamp or SCN (System Change Number) specified in the AS OF clause determines the point in time to which the query should be flashed back. As long as the required undo data is available within the retention period, the query will be successful.
- To ensure that Flashback Queries can be executed for the desired duration, you should configure appropriate values for UNDO_RETENTION and monitor the undo tablespace size to prevent it from becoming full prematurely. Additionally, periodic backups can help preserve historical data beyond the retention period specified for undo data.
- ☞ By setting the UNDO_RETENTION parameter appropriately and ensuring that the undo tablespace is adequately sized, you effectively control the retention period for Flashback Queries in Oracle. Flashback Queries will be able to access historical data within the retention period specified by UNDO_RETENTION.

View Undo Space Information in oracle

- ☞ In Oracle, you can view information about undo space usage and undo tablespace using various data dictionary views. Here are some commonly used views to retrieve undo space information:
- **DBA_UNDO_EXTENTS:** This view provides information about the extents allocated for undo segments in the database. You can query this view to see the size and status of undo extents.
 - **DBA_UNDO_USAGE:** This view displays the undo space usage statistics for each undo tablespace. It shows the amount of undo space used, the maximum amount of undo space used, and the number of transactions using undo space.

- **V\$UNDOSTAT:** This dynamic performance view provides statistical information about undo space usage over time. You can use this view to monitor undo space consumption trends.
 - **V\$TRANSACTION:** While not specifically for undo space information, this view can be used to monitor active transactions in the database, which indirectly affects undo space usage.
- ☞ By querying these views, you can obtain insights into undo space usage and monitor the health of your undo tablespaces.

Using the Undo Advisor

- The Undo Advisor in Oracle is a feature that helps database administrators to estimate the optimal size for the undo tablespace based on the workload of the database. This feature is particularly useful for databases where the workload varies over time and it's challenging to determine the appropriate size for the undo tablespace manually.
- Automatic tuning of undo retention typically achieves better results with a fixed-size undo tablespace. If you decide to use a fixed-size undo tablespace, the Undo Advisor can help you estimate needed capacity.
- You can access the Undo Advisor through Oracle Enterprise Manager (EM) or through the **DBMS_ADVISOR** PL/SQL package or through the **DBMS_UNDO_ADV** PL/SQL package.
- The Undo Advisor relies for its analysis on data collected in the Automatic Workload Repository (AWR). Thus, it is important that the AWR have adequate workload statistics available so that the Undo Advisor can make accurate recommendations. For newly created databases, adequate statistics may not be available immediately. In such cases, continue to use the default auto-extending undo tablespace until at least one workload cycle completes.
- Once you've accessed the Undo Advisor (whichever method you choose), the process typically involves these steps:
 - **Specify Advisor Parameters:** Depending on the chosen method, you might need to provide details like the undo tablespace name or desired confidence level for recommendations.

- **Run the Analysis:** Initiate the advisor to analyze undo usage patterns and workload statistics from the Automatic Workload Repository (AWR).
- **Review Recommendations:** The advisor will suggest an optimal undo tablespace size and minimum undo retention period based on its analysis.
- **Implement Recommendations:** You can implement the advisor's suggestions by resizing the undo tablespace (using ALTER TABLESPACE) and adjusting the UNDO_RETENTION initialization parameter if needed.

Benefits of Using the Undo Advisor:

- *Improved Sizing:* The advisor helps you determine the appropriate undo tablespace size to accommodate your workload's undo requirements, preventing space exhaustion or excessive wasted space.
- *Optimized Retention:* It recommends a minimum undo retention period that balances the need for historical data access for Flashback operations with space efficiency.
- *Reduced Administration:* The advisor automates much of the analysis and recommendation process, simplifying undo management.

DBMS_UNDO_ADVISOR package

- The DBMS_UNDO_ADVISOR package provides subprograms to manage the execution of the Undo Advisor feature. Following table summarizes the DBMS_UNDO_ADV Package Subprograms:

Subprogram	Description
BEST_POSSIBLE_RETENTION Function	Returns the best possible value for the undo_retention parameter that the current undo tablespace can satisfy in order to maximize the usage of the current undo tablespace based on the historical information of the given period.
LONGEST_QUERY Function	This function returns the duration of the longest query, in seconds, for a given period.

<u>RBU MIGRATION</u> Function	Estimates the undo tablespace needed for migration from manual to automatic undo management.
<u>REQUIRED RETENTION</u> Function	Returns the required value for the <code>undo_retention</code> parameter to satisfy the longest query based on undo statistics available for a given period.
<u>REQUIRED UNDO SIZE</u> Function	Returns the required undo tablespace size in MB to satisfy a certain undo retention value based on undo statistics available for a given period.
<u>UNDO ADVISOR</u> Function	Uses the advisor framework to check if there is any problem with the current instance. It also provides recommendations.
<u>UNDO AUTOTUNE</u> Function	Determines whether auto-tuning of undo retention is enabled for the current undo tablespace.
<u>UNDO HEALTH</u> Function	Checks whether there is any problem with the current setting of undo retention and undo tablespace size based on the historical information of a given period and provides recommendations to fix the problem.
<u>UNDO INFO</u> Function	Retrieves information about the undo tablespace of the current instance.

BEST_POSSIBLE_RETENTION Function

This function returns the best possible value for the `undo_retention` parameter that the current undo tablespace can satisfy in order to maximize the usage for the current undo tablespace based on the historical information of a given period.

Viewing the output using the historical information in memory:

```
DBMS_UNDO_ADV.BEST_POSSIBLE_RETENTION()
RETURN NUMBER;
```

Viewing the output using start time and end time:

```
DBMS_UNDO_ADV.BEST_POSSIBLE_RETENTION(    start_time    IN    DATE,
end_time    IN DATE) RETURN NUMBER;
```

REQUIRED_RETENTION Function

— This function returns the required value for the undo_retention parameter to satisfy the longest query based on undo statistics available for a given period.

Viewing the output using the historical information in memory:

```
DBMS_UNDO_ADV.REQUIRED_RETENTION()
RETURN NUMBER;
```

Viewing the output using start time and end time:

```
DBMS_UNDO_ADV.REQUIRED_RETENTION(
    start_time IN DATE,
    end_time   IN DATE)
RETURN NUMBER;
```

UNDO_ADVISOR Function

This function uses the advisor framework to check if there is any problem with the current instance and provide recommendations.

→ Note: This function should be used when undo_management is set to auto.

Viewing the output using the historical information in memory:

```
DBMS_UNDO_ADV.UNDO_ADVISOR(
    instance_id IN NUMBER)
RETURN VARCHAR2;
```

Viewing the output using start time and end time:

```

DBMS_UNDO_ADV.UNDO_ADVISOR(
    start_time IN DATE,
    end_time    IN DATE,
    instance_id IN NUMBER)

RETURN VARCHAR2;

```

LONGEST_QUERY Function

— This function returns the duration of the longest query, in seconds, for a given period.

Viewing the output using the historical information in memory:

```

DBMS_UNDO_ADV.LONGEST_QUERY()

RETURN NUMBER;

```

Viewing the output using start time and end time:

```

DBMS_UNDO_ADV.LONGEST_QUERY(
    start_time IN DATE,
    end_time    IN DATE)

RETURN NUMBER;

```

REQUIRED_UNDO_SIZE Function

This function returns the required undo tablespace size (in MB) to satisfy certain undo retention value based on undo statistics available for a given period.

Note: Zero will be returned if the information about the given period is not available.

Viewing the output using the historical information in memory:

```

DBMS_UNDO_ADV.REQUIRED_UNDO_SIZE(
    retention IN NUMBER)

RETURN NUMBER;

```

Viewing the output using start time and end time:

```
DBMS_UNDO_ADV.REQUIRED_UNDO_SIZE(
    retention IN NUMBER,
    start_time IN DATE,
    end_time IN DATE)

RETURN NUMBER;
```

UNDO_INFO Function

- This function retrieves information about the undo tablespace of the current instance.
- This function returns the undo tablespace name, maximum possible size for the undo tablespace, current undo retention value, and boolean values to verify if it is auto-extensible or if the undo tablespace has guaranteed undo retention.

```
DBMS_UNDO_ADV.UNDO_INFO(
    tbs_name OUT VARCHAR2,
    tbs_size OUT NUMBER,
    tbs_autoextend OUT BOOLEAN,
    tbs_retention OUT NUMBER,
    tbs_guarantee OUT BOOLEAN)
RETURN BOOLEAN;
```

Size the Undo Tablespace

- An appropriately sized undo tablespace is crucial for ensuring smooth database operation and enabling functionalities like Flashback queries for data recovery.
- Sizing the Undo Tablespace in Oracle depends on several factors such as the workload of the database, the frequency of transactions, and the duration of transactions.
- There's no one-size-fits-all solution for sizing the Undo Tablespace. It requires careful analysis of the specific database workload and continuous monitoring to ensure optimal performance.
- Here's a **general guideline to help you size the Undo Tablespace:**
 - ☞ **Estimate the Undo Space Requirement:** You need to estimate the maximum amount of undo space required during peak workload periods. This can be

calculated based on the number of concurrent transactions, the duration of transactions, and the amount of data modified by each transaction.

- ☞ **Monitor Undo Usage:** Use Oracle's performance monitoring tools to track the actual usage of the Undo Tablespace over a period of time. This will help you understand the peak usage and plan for capacity accordingly.
- ☞ **Consider Retention Period:** Determine the retention period for undo data. This is the duration for which Oracle should retain undo data before it is overwritten. This can be specified in terms of seconds or transactions.
- ☞ **Size the Undo Tablespace:** Based on the estimation from steps 1 and 2, and considering the retention period from step 3, size the Undo Tablespace to accommodate the maximum expected undo data during peak workload periods.
- ☞ **Allocate Sufficient Space:** Ensure that the Undo Tablespace has sufficient space to handle the workload without running out of space. It's a good practice to allocate extra space to account for unexpected spikes in workload.
- ☞ **Autoextend:** Consider enabling autoextend for the Undo Tablespace so that it can automatically grow if needed. However, be cautious with autoextend as it can lead to unexpected growth and potential disk space issues if not monitored carefully.
- ☞ **Regular Monitoring and Adjustment:** Continuously monitor the usage of the Undo Tablespace and adjust its size accordingly based on the actual workload patterns

Alter an Undo Tablespace to a Fixed Size

- ❖ You can alter an undo tablespace in Oracle to a fixed size using the ALTER TABLESPACE statement. This allows you to manage undo space allocation more precisely and potentially improve space efficiency.

Syntax:

```
ALTER TABLESPACE <undo_tablespace_name> RESIZE DATAFILE
'<datafile_path>' SIZE <new_size_in_units>;
```

Example:

```
ALTER TABLESPACE undotbs1 RESIZE DATAFILE '/u01/oracle/undotbs1.dbf' SIZE  
500M;
```

- This example resizes the datafile /u01/oracle/undotbs1.dbf associated with the undotbs1 undo tablespace to a fixed size of 500MB.

Important Considerations:

- ***Ensure Enough Space:*** Before resizing, verify there's enough disk space available to accommodate the new size.
- ***Autoextend Disabled:*** Ensure autoextend is disabled for the datafile before resizing to a fixed size. Otherwise, the datafile might automatically grow beyond the specified limit. You can disable autoextend using the ALTER DATABASE DATAFILE ... NOAUTOEXTEND statement.
- ***Impact on Existing Transactions:*** Resizing might affect ongoing transactions depending on their undo usage. It's recommended to perform the resize during a maintenance window with minimal database activity.
- ***Monitor Space Usage:*** Continuously monitor undo space consumption even after resizing. Your undo requirements might change over time, and you might need to adjust the size again in the future.

By following these steps and considering these points, you can successfully alter your undo tablespace to a fixed size in Oracle, providing more granular control over undo space allocation. Remember to choose a size that meets your undo retention needs while leaving some buffer space for potential future growth

Backup And Recovery

As a backup administrator, your principal duty is to devise, implement, and manage a backup and recovery strategy.

In general, the purpose of a *backup and recovery* strategy is to protect the database against data loss and reconstruct the database after data loss. Typically, backup administration tasks include the following:

- Planning and testing responses to different kinds of failures
- Configuring the database environment for backup and recovery

- Setting up a backup schedule
- Monitoring the backup and recovery environment
- Troubleshooting backup problems
- Recovering from data loss if the need arises

As a backup administrator, you may also be asked to perform other duties that are related to backup and recovery:

- Data archival, which involves creating a database copy for long-term storage
- Data transfer, which involves moving data from one database or one host to another

NOTES:

- A **backup** is a copy of data of a database that you can use to reconstruct data. A backup can be either a physical backup or a logical backup.
- **Physical backups** are copies of the physical files used in storing and recovering a database. These files include data files, control files, and archived redo logs. Ultimately, every physical backup is a copy of files that store database information to another location, whether on disk or on offline storage media such as tape.
- **Logical backups** contain logical data such as tables and stored procedures. You can use Oracle Data Pump to export logical data to binary files, which you can later import into the database. The Data Pump command-line clients expdp and impdp use the DBMS_DATAPUMP and DBMS_METADATA PL/SQL packages.
- Physical backups are the foundation of any sound backup and recovery strategy. Logical backups are a useful supplement to physical backups in many circumstances but are not sufficient protection against data loss without physical backups.
- Unless otherwise specified, the term backup as used in the backup and recovery documentation refers to a physical backup. Backing up a database is the act of making a physical backup.

Oracle Backup and Recovery Solutions

Oracle provides multiple solutions for performing backup and recovery. The following solutions are available when implementing a backup and recovery strategy:

❖ Recovery Manager (RMAN)

- Recovery Manager is fully integrated with the Oracle Database to perform a range of backup and recovery activities, including maintaining an RMAN repository of historical data about backups. You can access RMAN through the command line or through Oracle Enterprise Manager.

❖ Oracle Enterprise Manager Cloud Control

- Oracle Enterprise Manager Cloud Control (Cloud Control) provides a graphical front end and scheduling facilities for RMAN. You enter job parameters, specify a job schedule, and Cloud Control runs RMAN to conduct the backup and recovery operations.

❖ Zero Data Loss Recovery Appliance (Recovery Appliance)

- Recovery Appliance is a cloud-scale Engineered System that provides data protection for all Oracle Databases in the enterprise. Integrated with RMAN and Cloud Control, the Recovery Appliance provides a single repository for backups of multiple databases.

❖ User-managed backup and recovery

- In this solution, you perform backup and recovery with a mixture of host operating system commands and SQL*Plus recovery commands. You are responsible for determining all aspects of when and how backups and recovery are done.

These solutions are supported by Oracle and are fully documented, but **RMAN is the preferred solution for database backup and recovery. RMAN provides a common interface for backup tasks across different host operating systems, and offers several backup techniques not available through user-managed methods**

Full Backup and Incremental Backup

- A full backup captures a complete snapshot of the entire Oracle database, including all datafiles, control files, and archived redo logs.
- Incremental backups capture only the changes made to the database since the last backup, whether it's a full backup or a previous incremental backup.
- Execute the RMAN command to perform a full backup, specifying the database or specific datafiles, control files, and archived redo logs to include in the backup.

`BACKUP DATABASE; -- Backs up the entire database`

- When performing incremental backups with RMAN, you need to specify the level of the incremental backup, either 0 or 1.

`BACKUP DATABASE INCREMENTAL LEVEL 0;`

`BACKUP DATABASE INCREMENTAL LEVEL 1;`

Level 0 Incremental Backup:

- This is the initial incremental backup taken after a full backup.
- It captures all changed data blocks since the last full backup, similar to a full backup but marked as level 0.
- It serves as the baseline for subsequent level 1 incremental backups.

Level 1 Incremental Backup:

- This captures only the data blocks that have changed since the last full backup or the most recent level 0 incremental backup.
- By default, level 1 backups are cumulative, meaning they capture all the changes since the most recent full or level 0 backup (acting like a roll-up of changes).

Backing Up the Database

- This sections discuss about how to perform the most basic backup tasks and implement backup strategies using **RMAN**.

About of RMAN Backups

- Oracle Recovery Manager (RMAN), a command-line tool, is the method preferred by Oracle for efficiently backing up and recovering your Oracle database. The files

protected by the backup and recovery facilities built into RMAN include data files, control files, server parameter files, and archived redo log files. With these files you can reconstruct your database. RMAN is designed to work intimately with the server, providing block-level corruption detection during backup and restore. RMAN optimizes performance and space consumption during backup with file multiplexing and backup set compression, and integrates with leading tape and storage media products.

- RMAN backups are created using the BACKUP command.
- The primary purpose of RMAN backups is to protect your data. If a media failure or disaster occurs, then you can restore your backups and recover lost changes.
- You can also make backups to preserve data for long-time archival and to transfer data.
- You can back up all or part of your database with the BACKUP command from within the RMAN client.
- In many cases, after your database has been configured in accordance with your backup strategy, you can back up the database by entering the following command at the RMAN prompt:
RMAN> BACKUP DATABASE;
- RMAN uses the configured settings, the records of previous backups, and the control file record of the database structure to determine an efficient set of steps for the backup. RMAN then performs these steps
- You can provide **arguments to the BACKUP command** to override the default backup options.
- If you specify only the minimum required options for an RMAN command such as BACKUP DATABASE, then RMAN automatically determines the destination device, locations for backup output, and a backup tag based on your configured environment and built-in RMAN defaults.

ARCHIVELOG and NOARCHIVELOG Mode

- ☑ One of the important decisions you need to make as a DBA is to determine if the database must be run in ARCHIVELOG mode or NOARCHIVELOG mode. The mode you choose depends on your availability and reliability requirements. It also impacts the type of backup and recovery operations that you can perform.

- ☑ In NOARCHIVELOG mode, the filled redo log groups that become inactive can be reused. This mode protects the database against instance failure, but not against media failure. In ARCHIVELOG mode, filled groups of redo logs are archived. This mode protects the database from both instance and media failure, but may require additional hardware resources.

RMAN Repository

- ☑ Oracle Recovery Manager (RMAN) maintains a record of database files and backups for each database on which it performs operations. This metadata is called the RMAN repository. When you request recovery of a database, RMAN uses the repository metadata to choose the most efficient backups needed for this restore and recovery.
- ☑ The primary location for the RMAN repository for a database is its control file. The importance of this metadata for RMAN is one more reason why protecting your control file is a vital part of your backup strategy. In some installations, a second copy of the RMAN repository is stored in a schema called the recovery catalog. The recovery catalog is located in a separate database and can store metadata for multiple target databases.
- ☑ It is recommended that you use a recovery catalog. Because a recovery catalog stores metadata history for longer than the control file, you can perform a recovery that goes further back in time than the history in the control file. Also, if the target control file and all backups are lost, then the RMAN metadata in the recovery catalog can be used.

Image Copies and Backup Sets

- ☑ Database backups created by Oracle Recovery Manager (RMAN) are stored as image copies or backup sets.
- ☑ **Image copies** are exact byte-for-byte copies of files. You can create an image copy by copying a file at the operating system level. Unlike copying files at the operating system level, however, image copies created through RMAN are recorded in the RMAN repository so that RMAN can use these copies during database restore operations and recovery. RMAN can restore files only if they are recorded in the RMAN repository. RMAN can create image copies only on disk.

- ☑ **Backup sets** are logical entities produced by the RMAN BACKUP command. This command can produce one or more backup sets on disk or tape devices. Although image copies cannot use all RMAN features, their advantages are that you can apply incremental backups to them (synthetic full backups) and you can use them directly in place without first copying them, for very fast restores.
- ☑ Each backup set contains one or more physical files called backup pieces. A backup piece stores the backup of one or more database files in a compact RMAN-specific format. One advantage of backup sets is that RMAN uses unused block compression to save space in backing up data files. Only those blocks in the data files that have been used to store data are included in the backup set. Backup sets can also be compressed, encrypted, sent to tape, and use advanced unused-space compression that is not available with datafile copies.

Full Backups and Incremental Backups

- ☑ A **full backup** of a data file includes all used blocks of the data file. A full backup can be either an image copy or backup set.
- ☑ An **incremental backup** copies only those blocks in a data file that change between backups.
 - A **level 0** incremental backup, which copies all blocks in the data file, is used as a starting point for an incremental backup strategy.
 - A **level 1** incremental backup copies only images of blocks that have changed since the previous level 0 or level 1 incremental backup. Level 1 backups can be cumulative, in which case all blocks changed since the most recent level 0 backup are included, or differential, in which case only blocks changed since the most recent level 0 or level 1 incremental backup are included.
 - Incremental backups at level 0 can be either backup sets or image copies, but incremental backups at level 1 can only be backup sets.
 - A typical incremental strategy makes level 1 backups at regular intervals such as once each day.
 - During recovery, Oracle Recovery Manager (RMAN) will automatically apply both incremental backups and redo logs as required, to recover the database to the exact point in time desired.

Consistent and Inconsistent Backups

- ☑ A backup is either consistent or inconsistent. To make a consistent backup, your database must have been shut down cleanly and remain closed for the duration of the backup. All committed changes are written to the data files during the shut down process, so the data files are in a transaction-consistent state. When you restore your data files from a consistent backup, you can open the database immediately.
- ☑ If the database is in ARCHIVELOG mode, then you can make inconsistent backups that are recoverable using archived redo log files. Open database backups are inconsistent because the online redo log files contain changes not yet applied to the data files. The online redo log files must be archived and then backed up with the data files to ensure recoverability.
- ☑ Despite the name, an inconsistent backup is as robust a form of backup as a consistent backup. The advantage of making inconsistent backups is that you can back up your database while the database is open for updates.

Specifying the Device Type for an RMAN Backup

- The BACKUP command takes a DEVICE TYPE clause that specifies whether to back up to disk or tape device.
 - When you run BACKUP without a DEVICE TYPE clause, RMAN stores the backup on the configured default device (disk or SBT). You set the default device with the CONFIGURE DEFAULT DEVICE TYPE command.
- [SBT = System Backup to Tape]

Example: This example illustrates a backup to disk.

BACKUP

DEVICE TYPE DISK

DATABASE;

Specifying Backup Set or Copy for an RMAN Backup to Disk

- RMAN can create backups on disk as image copies or as backup sets.

- If a default device has been configured, you can override this default with the AS COPY or AS BACKUPSET clauses.

Example: This example uses BACKUP AS COPY to back up to disk as image copies.

```
BACKUP AS COPY
    DEVICE TYPE DISK
    DATABASE;
```

- To back up your data into backup sets, use the AS BACKUPSET clause. Following example illustrates how you can allow backup sets to be created on the configured default device, or direct them specifically to disk or tape.

```
BACKUP AS BACKUPSET
    DATABASE;
```

```
BACKUP AS BACKUPSET
    DEVICE TYPE DISK
    DATABASE;
```

```
BACKUP AS BACKUPSET
    DEVICE TYPE SBT
    DATABASE;
```

NOTES:

- The COPY command in RMAN is used to create physical copies of database files, which are identical duplicates of the original files. When you use the COPY command, RMAN reads the data blocks directly from the database

files and writes them to the specified backup location without any modification or transformation.

- The BACKUPSET command in RMAN is used to create backups in the form of backup sets. A backup set is a logical grouping of database files and blocks that RMAN backs up together. Backup sets are stored in RMAN-specific binary format and contain only the used blocks, not the entire database files.
- Use COPY when you need exact replicas of database files, fast restore times, or if you require backups that can be directly accessed by operating system utilities.
- Use BACKUPSET when you prioritize storage efficiency, want the benefits of RMAN compression, or need advanced recovery options such as block-level recovery.

Specifying a Format for RMAN Backups

- RMAN provides a range of options to name the files generated by the BACKUP command.
- RMAN uses the following set of rules to determine the format of the output files, which are listed in order of precedence:
 - If a FORMAT parameter is specified on the BACKUP command, then this setting controls the generated file name.

For example, you can direct the output to a specific location, as shown in the following command:

```
BACKUP DATABASE
```

```
    FORMAT '+dgroup1'; # specifies an ASM disk group
```

- If a FORMAT setting is configured for the specific channel used for the backup, then this setting controls the generated file name.
- If a FORMAT setting is configured for the device type used for the backup, then this setting controls the generated file name.
- If a fast recovery area is enabled during a disk backup, and if FORMAT is not specified, then RMAN creates the backup with an automatically generated name in the fast recovery area.

- If no other conditions in this list apply, then the default location and file name format of the backup are platform-specific.

Specifying Multiple Formats for Disk Backups

- When backing up to disk, you can specify a format to spread the backup across several drives for improved performance.
- To back up a database to multiple disk drives, allocate one DISK channel for each disk drive and specify the format string on the ALLOCATE CHANNEL command so that the file names are on different disks.

```

RUN{

    ALLOCATE CHANNEL disk1 DEVICE TYPE DISK FORMAT '/disk1/%d_backups/%U';

    ALLOCATE CHANNEL disk2 DEVICE TYPE DISK FORMAT '/disk2/%d_backups/%U';

    ALLOCATE CHANNEL disk3 DEVICE TYPE DISK FORMAT '/disk3/%d_backups/%U';

    BACKUP AS COPY DATABASE;

}

```

- To create a default configuration that distributes backups to multiple disk drives by default, configure multiple disk channels.

Specifying Tags for an RMAN Backup

- RMAN attaches a character string called a tag to every backup it creates, as a way of identifying the backup. You can either accept the default tag or specify your own with the TAG parameter of the BACKUP command.

Displaying Backup Reports

- Backup reports contain summary and detailed information about past backup jobs run by Oracle Recovery Manager (RMAN). The view V\$RMAN_BACKUP_JOB_DETAILS contains information about backup jobs run by RMAN. This view contains information such as the time taken for the backup, when a job started and finished, and what type of backup was performed, and the status of the backup job.

To display backup reports:

Use the following query to display the backup job history.

```
SELECT
SESSION_KEY,INPUT_TYPE,STATUS,START_TIME,END_TIME,ELAPSED
_SECONDS/3600 hrs
FROM V$RMAN_BACKUP_JOB_DETAILS;
```

SESSION_KEY	INPUT_TYPE	STATUS	START_TIME	END_TIME	HRS
8	DB FULL	FAILED	27-MAR-12	27-MAR-12	1.64666666
50	DB FULL	COMPLETED	28-MAR-12	28-MAR-12	.243055555
69	DB FULL	COMPLETED	30-MAR-12	05-APR-12	147.176388

SESSION_KEY is the unique key for the RMAN session in which the backup job occurred

Restore and Recover

- In Oracle, "Restore" and "Recover" are two essential operations in the realm of database management, particularly in the context of disaster recovery and data protection.
- **Restore** focuses on bringing back copies of database files (datafiles, control files, server parameter file) from backups to their original locations or designated spots. This operation re-establishes the physical structure of the database as it existed at the time the backup was taken.
- **Recovery** deals with applying changes (transactions) that occurred since the backup was taken. This ensures the database reaches a consistent state, reflecting the data up to a specific point in time.
 - Recovering a database involves applying the changes from the redo logs (also known as transaction logs or archive logs) to bring the database to a consistent state. This process rolls forward through the redo logs to reapply all the committed transactions that occurred after the backup was taken and rollbacks any uncommitted transactions. The aim of recovery is to bring the database to a consistent state, ensuring that it reflects all transactions up to the point of failure or data loss.
- It's important to note that the specifics of restoring and recovering a database may vary based on factors such as the backup strategy s), the presence of a standby

database, and the Oracle Database version being used. Additionally, Oracle provides various tools and utilities (such as RMAN - Recovery Manager) to streamline and automate the backup, restore, and recovery processes.

- ☑ **Restoring** a database means copying the physical files that comprise the database from a backup medium, typically disk or tape, to their original or to new locations. Database **recovery** is the process of updating database files restored from a backup with the changes made to the database after the backup by applying incremental backups and redo logs to the restored files.

Performing Oracle Advised Recovery

- The Oracle advised recovery feature uses Data Recovery Advisor, which is an Oracle Database feature that automatically diagnoses data failures, determines and presents appropriate repair options, and performs repairs if requested by the user. By providing a centralized tool for automated data repair, Data Recovery Advisor improves the manageability and reliability of an Oracle database.

☞ **Note:** Data Recovery Advisor can only be used to diagnose and repair failures in multitenant container databases (CDBs). It is not supported for pluggable databases (PDBs).

- RMAN provides a command-line interface to the Data Recovery Advisor. You can use following RMAN commands to diagnose and repair data failures for the Oracle Database, including for Oracle RAC databases:
 - **LIST FAILURE**
Use this command to view problem statements for failures and the effect of these failures on database operations. Each failure is identified by a failure number.
 - **ADVISE FAILURE**
Use this command to view repair options, including both automated and manual repair options.
 - **REPAIR FAILURE**
Use this command to automatically repair failures listed by the most recent ADVISE FAILURE command.

About Data Recovery Advisor

In the context of Data Recovery Advisor, a health check is a diagnostic procedure run by the Health Monitor to assess the state of the database or its components. Health checks are invoked reactively when an error occurs. You can also invoke checks manually.

A **failure** is a persistent data corruption detected by a health check. Failures are usually detected reactively. A database operation involving corrupted data results in an error, which automatically invokes a health check in the database. The check searches the database for failures related to the error. If failures are diagnosed, then they are recorded in the Automatic Diagnostic Repository (ADR).

You can use Data Recovery Advisor to generate repair advice and repair failures only after failures have been detected by the database and stored in ADR. Data Recovery Advisor can report on and repair failures such as inaccessible files, physical and logical block corruptions, and I/O failures. Every failure has a failure priority: CRITICAL, HIGH, or LOW. Every failure also has a failure status of OPEN or CLOSED.

You can also use Data Recovery Advisor to view repair options. A **repair** is an action that fixes one or more failures. Examples of repairs include block media recovery, data file media recovery, and Oracle Flashback Database. Typically, Data Recovery Advisor presents both automated and manual repair options. If appropriate, you can choose an automated repair option in order to perform a repair. In this case, Data Recovery Advisor verifies the repair success, and closes the relevant repaired failures

Using Data Recovery Advisor

- The recovery process begins when you either suspect or discover a failure. You can discover failures in many ways, including error messages, alerts, trace files, and health checks. You can then use Data Recovery Advisor to gain information and advice about failures and repair them automatically.

To use the Oracle advised recovery strategy to automatically repair failures:

1. Connect Oracle Recovery Manager (RMAN) to the target database.
In a multitenant environment, connect to the root as a common user with the SYSDBA or SYSBACKUP privilege.
2. List all the failures know to the Data Recovery Advisor by running the following command:

```
LIST FAILURE;
```

List of Database Failures

```
=====
```

```
Failure ID Priority Status Time Detected Summary
```

```
-----
```

Failure ID	Priority	Status	Time Detected	Summary
142	HIGH	OPEN	23-APR-07	One or more non-system datafiles are missing
101	HIGH	OPEN	23-APR-07	Datafile 1: '/disk1/oradata/prod/system01.dbf' contains one or more corrupt blocks

Wherever possible, RMAN consolidates failures while displaying the result of the LIST FAILURE command. For example, if a data file contains multiple block failures, the LIST FAILURE command consolidates and displays the repair options.

3. If you suspect that some failures that have not been automatically diagnosed by the database exist, then check for corrupt blocks and missing data files by using the following command:

```
VALIDATE DATABASE;
```

If a problem is detected during the validation, then RMAN triggers the execution of a failure assessment

4. Determine repair options, both automatic and manual, by using the following command:

```
ADVISE FAILURE;
```

5. Fix the failures by using the following command:

```
REPAIR FAILURE;
```

Automated repairs are performed by the Data Recovery Advisor. In certain cases, such as when no backups exist for a lost control file, the only repair option possible is the manual option

NOTES

- ☞ Before you perform any operations using RMAN, you must connect to a target database.
- ☞ The RMAN client is started by issuing the `rman` command at the command prompt of your operating system. RMAN displays a prompt for your commands as shown in the following example:

```
% rman
```

```
RMAN>
```

- ☞ RMAN connections to a database are specified and authenticated in the same way as SQL*Plus connections to a database. The only difference is that RMAN connections to a target or auxiliary database require either the `SYSDBA` or `SYSBACKUP` privilege. Any user can be granted this privilege.
- ☞ You can connect to a database with command-line options or by using the `CONNECT TARGET` command. The following example starts RMAN and then connects to a target database through Oracle Net as user `sbu`, which is created with the `SYSBACKUP` privilege. RMAN prompts for a password.

```
% rman
```

```
RMAN> CONNECT TARGET "sbu@prod AS SYSBACKUP"
```

```
target database Password: password
```

```
connected to target database: PROD (DBID=39525561)
```

- ☞ To quit the RMAN client, enter `EXIT` at the RMAN prompt:

Performing User-Directed Recovery

User-Directed Recovery enables you to use flashback features and perform restore operations and recovery procedures. For example, you can do the following:

- Repair unwanted changes to database objects with the logical flashback features
- Rewind the entire database with Oracle Flashback Database
- Completely restore and recover the database
- Perform point-in-time recovery of the database or selected tablespaces
- Perform block media recovery of data files that have corrupted blocks

You can determine which parts of the database must be restored and recovered, including detecting situations such as corrupted database files before they affect database operations.

Overview of Oracle Flashback Technology

Oracle Flashback Technology is a group of Oracle Database features that let you view past states of database objects or to return database objects to a previous state without using point-in-time media recovery. With flashback features, you can:

- Perform queries that return past data
- Perform queries that return metadata that shows a detailed history of changes to the database
- Recover tables or rows to a previous point in time
- Automatically track and archive transactional data changes
- Roll back a transaction and its dependent transactions while the database remains online

Oracle Flashback features use the Automatic Undo Management (AUM) system to obtain metadata and historical data for transactions. They rely on undo data, which are records of the effects of individual transactions. For example, if a user runs an UPDATE statement to change a salary from 1000 to 1100, then Oracle Database stores the value 1000 in the undo data.

Undo data is persistent and survives a database shutdown. It is retained for the time specified by `undo_retention`, or up to the tuned undo retention in the presence of Automatic Undo Management (AUM). By using flashback features, you can use undo data to query past data or recover from logical damage. Besides using it in flashback features, Oracle Database uses undo data to perform these actions:

- Roll back active transactions
- Recover terminated transactions by using database or process recovery
- Provide read consistency for SQL queries

Notes:

- After executing a CREATE TABLE statement, wait at least 15 seconds to commit any transactions, to ensure that Oracle Flashback features (especially Oracle Flashback Version Query) reflect those transactions.
- Oracle Database recommends to avoid the usage of `versions_starttime`, `versions_endtime` or `scn_to_timestamp` columns in VERSIONS queries (including CTAS queries) to improve the performance.

Features of flashback technologies

- ☑ **Application Development Features**
- ☑ **Database Administration Features**

Application Development Features

- In application development, you can use these flashback features to report historical data or undo erroneous changes. (You can also use these features interactively as a database user or administrator.)

Oracle Flashback Query

- ☞ Use this feature to retrieve data for an earlier time that you specify with the AS OF clause of the SELECT statement.
- ☞ To use Oracle Flashback Query, use a SELECT statement with an AS OF clause. Oracle Flashback Query retrieves data as it existed at an

earlier time. The query explicitly references a past time through a time stamp or System Change Number (SCN). It returns committed data that was current at that point in time.

Uses of Oracle Flashback Query include:

- Recovering lost data or undoing incorrect, committed changes.
For example, if you mistakenly delete or update rows, and then commit them, you can immediately undo the mistake.
- Comparing current data with the corresponding data at an earlier time.
For example, you can run a daily report that shows the change in data from yesterday. You can compare individual rows of table data or find intersections or unions of sets of rows.
- Checking the state of transactional data at a particular time.
For example, you can verify the account balance of a certain day.
- Selecting data that was valid at a particular time or at any time within a user-defined valid time period.
For example, you can find employees with valid employee information as of a particular timestamp or between a specified start and end time in the specified valid time period.
- Simplifying application design by removing the need to store some kinds of temporal data.
Oracle Flashback Query lets you retrieve past data directly from the database.
- Applying packaged applications, such as report generation tools, to past data.
- Providing self-service error correction for an application, thereby enabling users to undo and correct their errors.

Example: Retrieving a Lost Row with Oracle Flashback Query

```
SELECT * FROM employees
AS OF TIMESTAMP
TO_TIMESTAMP('2004-04-04 09:30:00', 'YYYY-MM-DD HH:MI:SS')
WHERE last_name = 'Chung';
```

This retrieves the state of the record for Chung at 9:30AM, April 4, 2004.

Example: Restoring a Lost Row After Oracle Flashback Query

```
INSERT INTO employees (
    SELECT * FROM employees
    AS OF TIMESTAMP
    TO_TIMESTAMP('2004-04-04 09:30:00', 'YYYY-MM-DD HH:MI:SS')
    WHERE last_name = 'Chung'
);
```

This restores Chung's information to the employees table.

Oracle Flashback Version Query

- Use this feature to retrieve metadata and historical data for a specific time interval (for example, to view all the rows of a table that ever existed during a given time interval). Metadata for each row version includes start and end time, type of change operation, and identity of the transaction that created the row version. To create an Oracle Flashback Version Query, use the VERSIONS BETWEEN clause of the SELECT statement.
- Use Oracle Flashback Version Query to retrieve the different versions of specific rows that existed during a given time interval. A row version is created whenever a COMMIT statement is executed.
 - Specify Oracle Flashback Version Query using the VERSIONS BETWEEN clause of the SELECT statement. The syntax is either:

`VERSIONS BETWEEN { SCN | TIMESTAMP } start AND end`

where *start* and *end* are expressions representing the start and end, respectively, of the time interval to be queried. The time interval includes (*start* and *end*).

Or

`VERSIONS PERIOD FOR user_valid_time [ BETWEEN TIMESTAMP start AND end ]`

where *user_valid_time* refers to the user-specified valid time period,

- Oracle Flashback Version Query returns a table with a row for each version of the row that existed at any time during the specified time interval. Each row in the table includes pseudocolumns of metadata about the row version, which can reveal when and how a particular change (perhaps erroneous) occurred to your database.
- Following table describes the pseudocolumns of metadata about the row version. The `VERSIONS_*` pseudocolumns have values only for transaction-time Flashback Version Queries (that is, queries with the clause `BETWEEN TIMESTAMP start AND end`).

Pseudocolumn Name	Description
<code>VERSIONS_STARTSCN</code> <code>VERSIONS_STARTTIME</code>	Starting System Change Number (SCN) or <code>TIMESTAMP</code> when the row version was created. This pseudocolumn identifies the time when the data first had the values reflected in the row version. Use this pseudocolumn to identify the past target time for Oracle Flashback Table or Oracle Flashback Query. If this pseudocolumn is <code>NULL</code> , then the row version was created before <i>start</i> .
<code>VERSIONS_ENDSCN</code> <code>VERSIONS_ENDTIME</code>	SCN or <code>TIMESTAMP</code> when the row version expired. If this pseudocolumn is <code>NULL</code> , then either the row version was current at the time of the query or the row corresponds to a <code>DELETE</code> operation.
<code>VERSIONS_XID</code>	Identifier of the transaction that created the row version.
<code>VERSIONS_OPERATION</code>	Operation performed by the transaction: I for insertion, D for deletion, or U for update. The version is that of the row that was inserted, deleted, or updated; that is, the row after an <code>INSERT</code> operation, the row before

	a DELETE operation, or the row affected by an UPDATE operation. For user updates of an index key, Oracle Flashback Version Query might treat an UPDATE operation as two operations, DELETE plus INSERT, represented as two version rows with a D followed by an I VERSIONS_OPERATION.
--	---

Here is a typical use of Oracle Flashback Version Query:

```

SELECT versions_startscn, versions_starttime,

       versions_endscn, versions_endtime,

       versions_xid, versions_operation,

       last_name, salary

FROM employees

VERSIONS BETWEEN TIMESTAMP

       TO_TIMESTAMP('2008-12-18 14:00:00', 'YYYY-MM-DD HH24:MI:SS')

AND TO_TIMESTAMP('2008-12-18 17:00:00', 'YYYY-MM-DD HH24:MI:SS')

WHERE first_name = 'John';

```

- This query retrieves historical data for an employee named John in the employees table, focusing on changes that occurred between a specific time window. Essentially, this query retrieves the entire history (all versions) of the row for John (based on first_name) in the employees table, focusing on changes that happened between 2 PM and 5 PM on December 18th, 2008.

You can use VERSIONS_XID with Oracle Flashback Transaction Query to locate this transaction's metadata, including the SQL required to undo the row change and the user responsible for the change.

Flashback Version Query allows index-only access only with IOTs (index-organized tables), but index fast full scan is not allowed.

Oracle Flashback Transaction Query

Use this feature to retrieve metadata and historical data for a given transaction or for all transactions in a given time interval. To perform an Oracle Flashback Transaction Query, select from the static data dictionary view FLASHBACK_TRANSACTION_QUERY.

Typically, you use Oracle Flashback Transaction Query with an Oracle Flashback Version Query that provides the transaction IDs for the rows of interest.

Examples

- This statement queries the FLASHBACK_TRANSACTION_QUERY view for transaction information, including the transaction ID, the operation, the operation start and end SCNs, the user responsible for the operation, and the SQL code that shows the logical opposite of the operation:

```
SELECT xid, operation, start_scn, commit_scn, logon_user, undo_sql
FROM flashback_transaction_query

WHERE xid = HEXTORAW('000200030000002D');
```

- This statement uses Oracle Flashback Version Query as a subquery to associate each row version with the LOGON_USER responsible for the row data change:

```
SELECT xid, logon_user
FROM flashback_transaction_query
WHERE xid IN (
    SELECT versions_xid FROM employees VERSIONS BETWEEN TIMESTAMP
    TO_TIMESTAMP('2003-07-18 14:00:00', 'YYYY-MM-DD HH24:MI:SS') AND
    TO_TIMESTAMP('2003-07-18 17:00:00', 'YYYY-MM-DD HH24:MI:SS')
);
```

Using Oracle Flashback Transaction Query with Oracle Flashback Version Query

- In this example, a database administrator does this:

```
DROP TABLE emp;
```

```
CREATE TABLE emp (
    empno    NUMBER PRIMARY KEY,
    empname  VARCHAR2(16),
    salary   NUMBER
);
```

```
INSERT INTO emp (empno, empname, salary) VALUES (111, 'Mike', 555);
COMMIT;
```

```
DROP TABLE dept;
```

```
CREATE TABLE dept (
    deptno   NUMBER,
    deptname VARCHAR2(32)
);
```

```
INSERT INTO dept (deptno, deptname) VALUES (10, 'Accounting');
COMMIT;
```

- Now *emp* and *dept* have one row each. In terms of row versions, each table has one version of one row. Suppose that an erroneous transaction deletes *empno* 111 from table *emp*:

```
UPDATE emp SET salary = salary + 100 WHERE empno = 111;
INSERT INTO dept (deptno, deptname) VALUES (20, 'Finance');
DELETE FROM emp WHERE empno = 111;
COMMIT;
```

- Next, a transaction reinserts *empno* 111 into the *emp* table with a new employee name:

```
INSERT INTO emp (empno, empname, salary) VALUES (111, 'Tom', 777);
UPDATE emp SET salary = salary + 100 WHERE empno = 111;
UPDATE emp SET salary = salary + 50 WHERE empno = 111;
COMMIT;
```

- The database administrator detects the application error and must diagnose the problem. The database administrator issues this query to retrieve versions of the rows in the *emp* table that correspond to *empno* 111. The query uses Oracle Flashback Version Query *pseudocolumns*:

```
SELECT versions_xid XID, versions_startscn START_SCN,
       versions_endscn END_SCN, versions_operation OPERATION,
       empname, salary
FROM emp
VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE
WHERE empno = 111;
```

- Results are similar to:

XID	START_SCN	END_SCN	O	EMPNAME	SALARY
-----	-----	-----	-	-----	-----
--					
09001100B2200000	10093466		I	Tom	927
030002002B210000	10093459		D	Mike	555
0800120096200000	10093375	10093459	I	Mike	555

3 rows selected.

- The results table rows are in descending chronological order. The third row corresponds to the version of the row in the table *emp* that was inserted in the table when the table was created. The second row corresponds to the row in *emp* that the erroneous transaction deleted. The first row corresponds to the version of the row in *emp* that was reinserted with a new employee name.
- The database administrator identifies transaction 030002002B210000 as the erroneous transaction and uses Oracle Flashback Transaction Query to audit all changes made by this transaction:

```
SELECT xid, start_scn, commit_scn, operation, logon_user, undo_sql
FROM flashback_transaction_query
WHERE xid = HEXTORAW('000200030000002D');
```

- Results are similar to:

XID	START_SCN	COMMIT_SCN	OPERATION	LOGON_USER

UNDO_SQL				

030002002B210000	10093452	10093459	DELETE	HR
insert into "HR"."EMP"("EMPNO","EMPNAME","SALARY") values ('111','Mike','655');				
030002002B210000	10093452	10093459	INSERT	HR
delete from "HR"."DEPT" where ROWID = 'AAATjuAAEAAAAJrAAB';				
030002002B210000	10093452	10093459	UPDATE	HR
update "HR"."EMP" set "SALARY" = '555' where ROWID = 'AAATjsAAEAAAAJ7AAA';				
030002002B210000	10093452	10093459	BEGIN	HR

4 rows selected.

- To make the result of the next query easier to read, the database administrator uses these SQL*Plus commands:

```
COLUMN operation FORMAT A9
```

```
COLUMN table_name FORMAT A10
```

```
COLUMN table_owner FORMAT A11
```

- To see the details of the erroneous transaction and all subsequent transactions, the database administrator performs this query:

```
SELECT  xid,  start_scn,  commit_scn,  operation,  table_name,
        table_owner
```

```
FROM flashback_transaction_query
```

```
WHERE table_owner = 'HR'
```

```
AND start_timestamp >=
```

```
TO_TIMESTAMP ('2002-04-16 11:00:00','YYYY-MM-DD HH:MI:SS');
```

☞ Results are similar to:

XID	START_SCN	COMMIT_SCN	OPERATION	TABLE_NAME	TABLE_OWNER
-----	-----	-----	-----	-----	-----
02000E0074200000	10093435	10093446	INSERT	DEPT	HR
030002002B210000	10093452	10093459	DELETE	EMP	HR
030002002B210000	10093452	10093459	INSERT	DEPT	HR
030002002B210000	10093452	10093459	UPDATE	EMP	HR
0800120096200000	10093374	10093375	INSERT	EMP	HR
09001100B2200000	10093462	10093466	UPDATE	EMP	HR
09001100B2200000	10093462	10093466	UPDATE	EMP	HR
09001100B2200000	10093462	10093466	INSERT	EMP	HR

8 rows selected.

DBMS_FLASHBACK Package

Use this feature to set the internal Oracle Database clock to an earlier time so that you can examine data that was current at that time, or to roll back a transaction and its dependent transactions while the database remains online.

The DBMS_FLASHBACK package provides the same functionality as Oracle Flashback Query, but Oracle Flashback Query is sometimes more convenient.

The DBMS_FLASHBACK package acts as a time machine: you can turn back the clock, perform normal queries as if you were at that earlier time, and then return to the present. Because you can use the DBMS_FLASHBACK package to perform queries on past data without special clauses such as AS OF or VERSIONS BETWEEN, you can reuse existing PL/SQL code to query the database at earlier times.

You must have the EXECUTE privilege on the DBMS_FLASHBACK package.

To use the DBMS_FLASHBACK package in your PL/SQL code:

1. Specify a past time by invoking either
DBMS_FLASHBACK.ENABLE_AT_TIME or
DBMS_FLASHBACK.ENABLE_AT_SYSTEM_CHANGE_NUMBER.

2. Perform regular queries (that is, queries without special flashback-feature syntax such as AS OF). Do not perform DDL or DML operations.

The database is queried at the specified past time.

3. Return to the present by invoking DBMS_FLASHBACK.DISABLE.

You must invoke DBMS_FLASHBACK.DISABLE before invoking DBMS_FLASHBACK.ENABLE_AT_TIME or DBMS_FLASHBACK.ENABLE_AT_SYSTEM_CHANGE_NUMBER again. You cannot nest enable/disable pairs.

To use a cursor to store the results of queries, open the cursor before invoking DBMS_FLASHBACK.DISABLE. After storing the results and invoking DBMS_FLASHBACK.DISABLE, you can:

- Perform INSERT or UPDATE operations to modify the current database state by using the stored results from the past.
- Compare current data with the past data. After invoking DBMS_FLASHBACK.DISABLE, open a second cursor. Fetch from the first cursor to retrieve past data; fetch from the second cursor to retrieve current data. You can store the past data in a temporary table and then use set operators such as MINUS or UNION to contrast or combine the past and current data.

You can invoke DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER at any time to get the current System Change Number (SCN). DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER returns the current SCN regardless of previous invocations of DBMS_FLASHBACK.ENABLE.

Using Flashback Version Query with DBMS_FLASHBACK

- You can enable DBMS_FLASHBACK for a session using the DBMS_FLASHBACK.ENABLE_AT_SYSTEM_CHANGE_NUMBER or DBMS_FLASHBACK.ENABLE_AT_TIME procedure.

The ENABLE_AT_SYSTEM_CHANGE_NUMBER and ENABLE_AT_TIME procedures are based on System Change Number (SCN). You can use Oracle Flashback Version Query (version query) to retrieve committed row versions pertaining to a time interval. To use a version query (with the VERSIONS BETWEEN clause), you need to

specify the lower and upper SCN limits, either using specific SCN values or using the MINVALUE and MAXVALUE keywords.

- Using a version query along with DBMS_FLASHBACK affects the MAXVALUE keyword or the upper SCN limit in the version query, in that if the version query's upper SCN limit (through the MAXVALUE keyword or provided value) exceeds the DBMS_FLASHBACK SCN, the version query takes the DBMS_FLASHBACK SCN as the maximum limit. For example, suppose that the session-level Flashback is enabled using the ENABLE_AT_SYSTEM_CHANGE_NUMBER procedure with the SCN parameter set as SCN_X. A version query using VERSIONS BETWEEN clause is passed later with the minimum and maximum limits as SCN_A and SCN_B. When the version query runs, the SCNs are treated in the following manner:
 - When the versions query specifies a MAXVALUE clause (instead of a specific SCN_B), SCN_X is treated as the maximum limit.
 - When the versions query has two SCNs, SCN_A and SCN_B, and SCN_B is less than SCN_X, then the SCNs in the versions query are retained without any changes.
 - When the versions query has two SCNs, SCN_C and SCN_D, and SCN_D is greater than SCN_X, then SCN_X is treated as the maximum limit.
- The following code sample shows how a version query could be used after DBMS_FLASHBACK is enabled for a session.


```
DBMS_FLASHBACK.ENABLE_AT_SYSTEM_CHANGE_NUMBER(123);
...
...
SELECT * FROM <table_name> VERSIONS BETWEEN SCN 120 and 125;
...
DBMS_FLASHBACK.DISABLE();
```
- Here, the upper SCN limit of 125 in the version query is replaced with DBMS_FLASHABCK SCN of 123 because the version query's upper SCN limit is greater than the DBMS_FLASHABCK SCN. Therefore, the following is the resultant version query:


```
SELECT * FROM <table_name> VERSIONS BETWEEN SCN 120 and 123;
```

Flashback Transaction

Use Flashback Transaction to roll back a transaction and its dependent transactions while the database remains online. This recovery operation uses undo data to create and run the corresponding compensating transactions that return the affected data to its original state. (Flashback Transaction is part of DBMS_FLASHBACK package).

The DBMS_FLASHBACK.TRANSACTION_BACKOUT procedure rolls back a transaction and its dependent transactions while the database remains online. This recovery operation uses undo data to create and run the compensating transactions that return the affected data to its original state.

The transactions being rolled back are subject to these restrictions:

- They cannot have performed DDL operations that changed the logical structure of database tables.
- They cannot use Large Object (LOB) Data Types:
 - BFILE
 - BLOB
 - CLOB
 - NCLOB
- They cannot use features that LogMiner does not support.

The features that LogMiner supports depends on the value of the COMPATIBLE initialization parameter for the database that is rolling back the transaction. The default value is the release number of the most recent major release.

Flashback Transaction inherits SQL data type support from LogMiner. Therefore, if LogMiner fails due to an unsupported SQL data type in a the transaction, Flashback Transaction fails too.

Some data types, though supported by LogMiner, do not generate undo information as part of operations that modify columns of such types. Therefore, Flashback Transaction does not support tables containing these data types. These include tables with BLOB, CLOB and XML type.

[**LogMiner** is a powerful tool used in Oracle database environments for analyzing and recovering changes made to the database. It enables database administrators to query online and archived redo log files to extract valuable information about changes to the database, such as insert, update, and delete operations.]

Dependent Transactions

In the context of Flashback Transaction, transaction 2 can depend on transaction 1 in any of these ways:

Write-after-write dependency

Transaction 1 changes a row of a table, and later transaction 2 changes the same row.

Primary key dependency

A table has a primary key constraint on column c. In a row of the table, column c has the value v. Transaction 1 deletes that row, and later transaction 2 inserts a row into the same table, assigning the value v to column c.

Foreign key dependency

In table b, column b1 has a foreign key constraint on column a1 of table a. Transaction 1 changes a value in a1, and later transaction 2 changes a value in b1.

TRANSACTION BACKOUT Parameters

The parameters of the TRANSACTION_BACKOUT procedure are:

- Number of transactions to be backed out
- List of transactions to be backed out, identified either by name or by XID
- Time hint, if you identify transactions by name
- Specify a time that is earlier than any transaction started.
- Backout options as in following table

TRANSACTION_BACKOUT analyzes the transactional dependencies, performs DML operations, and generates reports. TRANSACTION_BACKOUT does not commit the DML operations that it performs as part of transaction backout, but it holds all the required locks on rows and tables in the right form, preventing other dependencies from entering the system. To make the transaction backout permanent, you must explicitly commit the transaction.

Option	Description
CASCADE	Backs out specified transactions and all dependent transactions in a post-order fashion (that is, children are backed out before parents are backed out). Without CASCADE, if any dependent transaction is not specified, an error occurs.
NOCASCADE	Default. Backs out specified transactions, which are expected to have no dependent transactions. First dependent transactions causes an error and appears in *_FLASHBACK_TXN_REPORT.
NOCASCADE_FORCE	Backs out specified transactions, ignoring dependent transactions. Server runs undo SQL statements for specified transactions in reverse order of commit times. If no constraints break and you are satisfied with the result, you can commit the changes; otherwise, you can roll them back.
NONCONFLICT_ONLY	Backs out changes to nonconflicting rows of the specified transactions. Database remains consistent, but transaction atomicity is lost.

TRANSACTION BACKOUT Reports

To see the reports that TRANSACTION_BACKOUT generates, query the static data dictionary views *_FLASHBACK_TXN_STATE and *_FLASHBACK_TXN_REPORT.

- The static data dictionary view *_FLASHBACK_TXN_STATE shows whether a transaction is active or backed out. If a transaction appears in this view, it is backed out.
- The static data dictionary view *_FLASHBACK_TXN_REPORT provides a detailed report for each backed-out transaction.

Using Flashback Time Travel

- Flashback Time Travel provides the ability to track and store definitional (including schema) and transactional changes to a table over its lifetime.
- Using Flashback Time Travel, you can enable tracking of DML (such as INSERT and DELETE) and DDL operations (such as creating and truncating tables) on a table that is being tracked (also called tracked table). You can then use Flashback Data Archive (also called Flashback Archive) to archive the changes made to the rows of the tracked table in history tables. Flashback Time Travel also maintains a history of the evolution of a table schema. Having the history of the table and schema enables you to issue flashback queries (AS OF and VERSIONS) on the table and its schema. You can also view the history of DDL and DML changes made to the table.
- With the Flashback Time Travel feature, you can create several Flashback Archives in your database. A Flashback Archive is a logical entity that is associated with a set of tablespaces. There is a quota that is reserved for the archives on those tablespaces and a retention period for the archived data. Using a Flashback Archive improves performance and helps in complying with record stage policies and audit reports.
- A Flashback Archive consists of one or more tablespaces or parts thereof. You can have multiple Flashback Archives. If you are logged on as SYSDBA, you can specify a default Flashback Archive for the system. A Flashback Archive is configured with retention time. Data archived in the Flashback Archive is retained for the retention time that is specified when creating the Flashback Archive.
- When choosing a Flashback Archive for a specific table, consider the data retention requirements for the table and the retention times of the Flashback Archives on which you have the FLASHBACK ARCHIVE object privilege.
- To ensure data security of the Flashback Archive history tables, when enabling a Flashback Archive for a table, you can specify that a tracked table should use a blockchain history table for the Flashback Archive.
- You can also use Flashback Time Travel in various scenarios, such as enforcing digital shredding, accessing historical data, selective data recovery, and auditing.

Database Administration Features

These flashback features are primarily for data recovery. Typically, you use these features only as a database administrator.

Oracle Flashback Table

Use this feature to restore a table to its state at a previous point in time. You can restore a table while the database is on line, undoing changes to only the specified table.

Oracle Flashback Drop

Use this feature to recover a dropped table. This feature reverses the effects of a DROP TABLE statement.

Oracle Flashback Database

Use this feature to quickly return the database to an earlier point in time, by using the recovery area. This is fast, because you do not have to restore database backups.

Configuring Your Database for Oracle Flashback Technology

Before you can use flashback features in your application, you or your database administrator must perform the configuration tasks as below:

- *Configuring Your Database for Automatic Undo Management*
- *Configuring Your Database for Oracle Flashback Transaction Query*
- *Configuring Your Database for Flashback Transaction*
- *Enabling Oracle Flashback Operations on Specific LOB Columns*
- *Granting Necessary Privileges*

Configuring Your Database for Automatic Undo Management

To configure your database for Automatic Undo Management (AUM), you or your database administrator must:

- Create an undo tablespace with enough space to keep the required data for flashback operations.

- The more often users update the data, the more space is required. The database administrator usually calculates the space requirement.
- Enable AUM. Set these database initialization parameters:
 - UNDO_MANAGEMENT
 - UNDO_TABLESPACE

For a fixed-size undo tablespace, Oracle Database automatically tunes the system to give the undo tablespace the best possible undo retention.

For an automatically extensible undo tablespace, Oracle Database retains undo data longer than the longest query duration and the low threshold of undo retention specified by the UNDO_RETENTION parameter.

Setting UNDO_RETENTION does not guarantee that unexpired undo data is not discarded. If the system needs more space, Oracle Database can overwrite unexpired undo with more recently generated undo data.

- Specify the RETENTION GUARANTEE clause for the undo tablespace to ensure that unexpired undo data is not discarded.

Configuring Your Database for Oracle Flashback Transaction Query

To configure your database for the Oracle Flashback Transaction Query feature, you or your database administrator must:

- Ensure that Oracle Database is running with version 10.0 compatibility.
- Enable supplemental logging:

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;
```

Configuring Your Database for Flashback Transaction

To configure your database for the Flashback Transaction feature, you or your database administrator must:

- With the database mounted but not open, enable ARCHIVELOG

```
ALTER DATABASE ARCHIVELOG;
```

- Open at least one archive log:

```
ALTER SYSTEM ARCHIVE LOG CURRENT;
```

- If not done, enable minimal and primary key supplemental logging:

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;
```

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA (PRIMARY KEY) COLUMNS;
```

- If you want to track foreign key dependencies, enable foreign key supplemental logging:

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA (FOREIGN KEY) COLUMNS;
```

Enabling Oracle Flashback Operations on Specific LOB Columns

- To enable flashback operations on specific LOB columns of a table, use the ALTER TABLE statement with the RETENTION option.
- Because undo data for LOB columns can be voluminous, you must define which LOB columns to use with flashback operations.

Granting Necessary Privileges

- You or your database administrator must grant privileges to users, roles, or applications that must use these flashback features.

For Oracle Flashback Query and Oracle Flashback Version Query

☞ To allow access to specific objects during queries, grant FLASHBACK and either READ or SELECT privileges on those objects.

☞ To allow queries on all tables, grant the FLASHBACK ANY TABLE privilege.

For Oracle Flashback Transaction Query

☞ Grant the SELECT ANY TRANSACTION privilege.

- ☞ To allow execution of undo SQL code retrieved by an Oracle Flashback Transaction Query, grant SELECT, UPDATE, DELETE, and INSERT privileges for specific tables.

For DBMS_FLASHBACK Package

- ☞ To allow access to the features in the DBMS_FLASHBACK package, grant the EXECUTE privilege on DBMS_FLASHBACK.

For Flashback Time Travel

- ☞ To allow a specific user to enable Flashback Time Travel on tables, using a specific Flashback Archive, grant the FLASHBACK ARCHIVE object privilege on that Flashback Archive to that user. To grant the FLASHBACK ARCHIVE object privilege, you must either be logged on as SYSDBA or have FLASHBACK ARCHIVE ADMINISTER system privilege.
- ☞ To allow execution of these statements, grant the FLASHBACK ARCHIVE ADMINISTER system privilege:
 - CREATE FLASHBACK ARCHIVE
 - ALTER FLASHBACK ARCHIVE
 - DROP FLASHBACK ARCHIVE
- ☞ To grant the FLASHBACK ARCHIVE ADMINISTER system privilege, you must be logged on as SYSDBA.
- ☞ To create a default Flashback Archive, using either the CREATE FLASHBACK ARCHIVE or ALTER FLASHBACK ARCHIVE statement, you must be logged on as SYSDBA.
- ☞ To disable Flashback Archive for a table that has been enabled for Flashback Archive, you must either be logged on as SYSDBA or have the FLASHBACK ARCHIVE ADMINISTER system privilege.

Types of Failure

- Several problems can halt the normal operation of an Oracle database or affect database I/O operations. The following sections describe the most common types of problems. For some of these problems, crash and instance recovery occur automatically and require no action on the part of the database administrator. For other problems, administrator-initiated media recovery is required.
- Types of failures are as follows
 - **Media Failure**
 - **Failure due to User Error**
 - **Database Instance Failure**
 - **Statement Failure**
 - **Process Failure**
 - **Network Failure**

Media Failure

- An error can occur when trying to write or read a file on disk that is required to operate an Oracle database. This occurrence is called media failure because there is a physical problem reading or writing to files on the storage medium.
- A common example of media failure is a disk head crash that causes the loss of all database files on a disk drive. All files associated with a database are vulnerable to a disk crash, including datafiles, control files, online redo logs, and archived logs.
- The appropriate recovery from a media failure depends on the files affected. Media failure is the primary concern of a backup and recovery strategy, because it typically requires restoring some or all database files and the application of redo during recovery.

Failure due to User Error

- As an administrator, you can do little to prevent user errors such as accidentally dropping a table. Often, user error can be reduced by increased training on database and application principles. You can also avoid user errors by administering privileges correctly so that users are able to do less potential damage. Furthermore,

by planning an effective recovery scheme ahead of time, you can ease the work necessary to recover from user errors.

- Typically, a user error such as a dropped table requires either re-entering the lost changes manually (if a record of them exists), importing the dropped object (if an export file exists), or performing incomplete recovery either of an individual tablespaces (called tablespace point-in-time recovery (TSPITR)) or of the entire database.

Database Instance Failure

- Database instance failure occurs when a problem prevents an Oracle database instance from continuing to run. An instance failure can result from a hardware problem, such as a power outage, or a software problem, such as an operating system crash. Instance failure also results when you issue a SHUTDOWN ABORT or STARTUP FORCE statement.

Statement Failure

- Statement failure occurs when there is a logical failure in the handling of a statement in an Oracle program. For example, assume that all extents of a table (in other words, the number of extents specified in the MAXEXTENTS parameter of the CREATE TABLE statement) are allocated, and are completely filled with data. A valid INSERT statement cannot insert a row because no space is available. Therefore, the statement fails.
- If a statement failure occurs, then the Oracle software or operating system returns an error. A statement failure usually requires no recovery steps: Oracle automatically corrects for statement failure by rolling back any effects of the statement and returning control to the application. The user can simply execute the statement again after the problem indicated by the error message is corrected. For example, if insufficient extents are allocated, then the DBA needs to allocate more extents so that the user's statement can execute

Process Failure

- A process failure is a failure in a user, server, or background process of a database instance such as an abnormal disconnect or process termination. When a process failure occurs, the failed subordinate process cannot continue work, although the other processes of the database instance can continue.
- The Oracle background process PMON detects aborted Oracle processes. If the aborted process is a user or server process, then PMON resolves the failure by rolling back the current transaction of the aborted process and releasing any resources that this process was using. Recovery of the failed user or server process is automatic. If the aborted process is a background process, then the instance usually cannot continue to function correctly. Therefore, you must shut down and restart the instance.

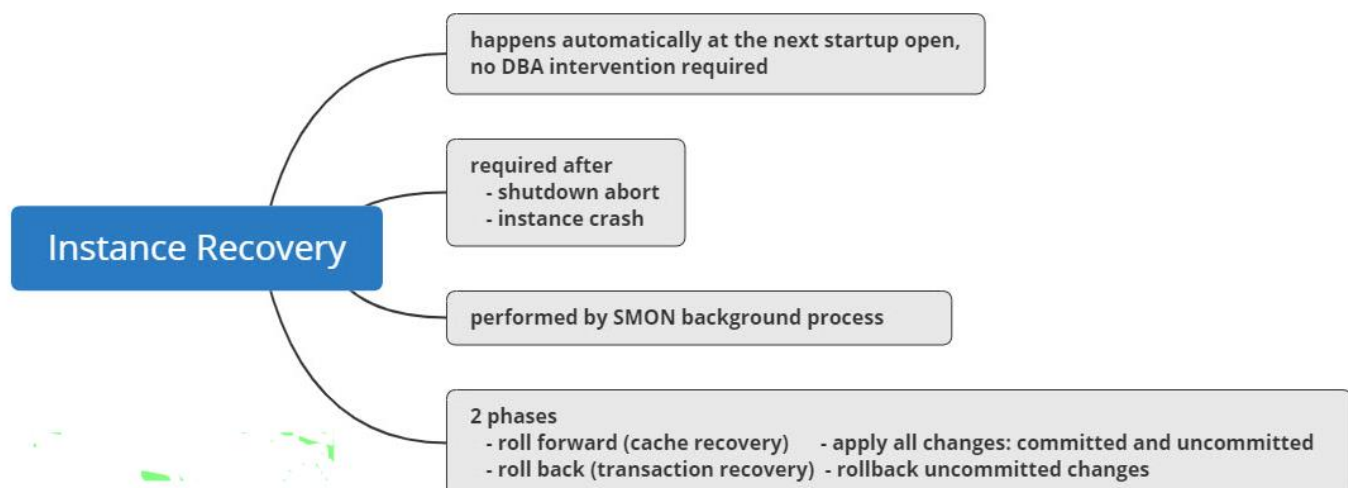
Network Failure

- When your system uses networks such as local area networks and phone lines to connect client workstations to database servers, or to connect several database servers to form a distributed database system, network failures such as aborted phone connections or network communication software failures can interrupt the normal operation of a database system. For example:
 - A network failure can interrupt normal execution of a client application and cause a process failure to occur. In this case, the Oracle background process PMON detects and resolves the aborted server process for the disconnected user process, as described in the previous section.
 - A network failure can interrupt the two-phase commit of a distributed transaction. After the network problem is corrected, the Oracle background process RECO of each involved database automatically resolves any distributed transactions not yet resolved at all nodes of the distributed database system

As new transactions encounter rows locked by dead transactions, they can automatically roll back the dead transaction to release the locks. If you are using Fast-Start Recovery, then only the data block is immediately rolled back, as opposed to the entire transaction.

Instance Recovery

- Database instance failure occurs when a problem prevents an Oracle database instance from continuing to run. An instance failure can result from a hardware problem, such as a power outage, or a software problem, such as an operating system crash. Instance failure also results when you issue a SHUTDOWN ABORT or STARTUP FORCE statement.
- Instance and crash recovery are the automatic application of redo log records to Oracle data blocks after a crash or system failure. If either a single instance database crashes or all instances of an OPS (Oracle Parallel Server) configuration crash, Oracle performs instance recovery at the next startup. If one or more instances of an OPS configuration crash, a surviving instance performs recovery.
- Instance and crash recovery occur in two phases.
 - In phase one, Oracle applies all committed and uncommitted changes in the redo log files to the affected datablocks.
 - In phase two, Oracle applies information in the rollback segments to undo changes made by uncommitted transactions to the data blocks.



Mechanics of Instance and Crash Recovery

When one or more instances fail, Oracle automatically recovers the lost changes associated with the instance or instances. Crash or instance recovery consists of the following steps:

1. Rolling forward to recover data that has not been recorded in the datafiles, yet has been recorded in the online redo log, including changes to undo blocks. This phase is called *cache recovery*.
2. Opening the database. Instead of waiting for all transactions to be rolled back before making the database available, Oracle allows the database to be opened as soon as cache recovery is complete. Any data that is not locked by unrecovered transactions is immediately available.
3. Marking all transactions systemwide that were active at the time of failure as DEAD and marking the rollback or undo segments containing these transactions as PARTLY AVAILABLE.
4. Rolling back dead transactions as part of SMON recovery. This phase is called *transaction recovery*.
5. Resolving any pending distributed transactions undergoing a two-phase commit at the time of the instance failure.
6. As new transactions encounter rows locked by dead transactions, they can automatically roll back the dead transaction to release the locks. If you are using Fast-Start Recovery, then only the data block is immediately rolled back, as opposed to the entire transaction.

To perform manual instance recovery or point-in-time recovery in Oracle, you can use Oracle Recovery Manager (RMAN) or SQL*Plus commands, depending on the specific requirements and circumstances of the recovery operation. The exact steps and commands involved will vary based on factors such as the nature of the failure, the backup strategy in place, and the desired recovery point. It's important to consult the Oracle documentation and follow best practices when performing manual recovery operations to ensure data integrity and minimize downtime.

Example:

```
RMAN> RECOVER DATABASE;
```

This command instructs RMAN to perform crash recovery on the database, applying the necessary redo logs to bring the database to a consistent state.

While instance recovery is typically automatic in Oracle 19c, there are scenarios where manual intervention may be necessary, such as incomplete or failed instance recovery or performing point-in-time recovery to restore the database to a specific SCN or timestamp.

Tune Instance Recovery

- Optimizing instance recovery time in Oracle ensures faster database restarts after crashes or failures.
- Tuning instance recovery in Oracle is crucial for minimizing downtime and improving the performance of the recovery process. Here are some strategies to tune instance recovery:
 - *Tuning the Duration of Instance and Crash Recovery*
 - *Monitoring Instance Recovery*
 - *Tuning the Phases of Instance Recovery*
 - *Transparent Application Failover*

Tuning the Duration of Instance and Crash Recovery

There are several methods for tuning instance and crash recovery to keep the duration of recovery within user-specified bounds. These methods are:

- *Using initialization parameters to influence the number of redo log records and data blocks involved in recovery*
- *Sizing the redo log file to influence checkpointing frequency*
- *Using SQL statements to initiate checkpoints*
- *Parallelizing instance recovery operations to further shorten the recovery duration*

Using Initialization Parameters to Influence Instance and Crash Recovery Time

During recovery, Oracle performs two main tasks:

- Read the redo logs to determine what has been changed

- Read data blocks to determine whether to apply changes

You can use three initialization parameters to influence how aggressively Oracle advances the checkpoint as shown in following table.

Table: Initialization Parameters Influencing Checkpoints

Parameter	Purpose
LOG_CHECKPOINT_TIMEOUT	Limit the number of seconds between the most recent redo record and the checkpoint.
LOG_CHECKPOINT_INTERVAL	Limit the number of redo records between the most recent redo record and the checkpoint.
FAST_START_IO_TARGET	Limit instance recovery time by controlling the number of data blocks Oracle processes during instance recovery.

Using SQL Statements to Initiate Checkpoints

- Besides setting initialization parameters and sizing your redo log files, you can also influence checkpoints with SQL statements. ALTER SYSTEM CHECKPOINT directs Oracle to record a checkpoint for the node, and ALTER SYSTEM CHECKPOINT GLOBAL directs Oracle to record a checkpoint for every node in a cluster.
- SQL-induced checkpoints are "heavyweight". This means Oracle records the checkpoint in a control file shared by all the redo threads. Oracle also updates the datafile headers. SQL-induced checkpoints move the checkpoint position to the point that corresponded to the end of the log when the command was initiated. These checkpoints can adversely affect performance because the additional writes to the datafiles increase system overhead.

Monitoring Instance Recovery

- Use the V\$INSTANCE_RECOVERY view to see your current recovery parameter settings. You can also use statistics from this view to calculate which parameter has the greatest influence on checkpointing. V\$INSTANCE_RECOVERY contains columns as shown in following table:

Table: V\$INSTANCE_RECOVERY

Column	Description
RECOVERY_ESTIMATED_IOS	The estimated number of data blocks to be processed during recovery based on the in-memory value of the fast-start checkpoint parameter.
ACTUAL_REDO_BKLS	The current number of redo blocks required for recovery.
TARGET_REDO_BKLS	The goal for the maximum number of redo blocks to be processed during recovery. This value is the minimum of the following 4 columns.
LOG_FILE_SIZE_REDO_BKLS	The number of redo blocks to be processed during recovery to guarantee that a log switch never has to wait for a checkpoint.
LOG_CHKPT_TIMEOUT_REDO_BKLS	The number of redo blocks that need to be processed during recovery to satisfy LOG_CHECKPOINT_TIMEOUT.
LOG_CHKPT_INTERVAL_REDO_BKLS	The number of redo blocks that need to be processed during recovery to satisfy LOG_CHECKPOINT_INTERVAL.
FAST_START_IO_TARGET_REDO_BKLS	The number of redo blocks that need to be processed during recovery to satisfy FAST_START_IO_TARGET.

The value appearing in the TARGET_REDO_BKLS column equals a value appearing in another column in the view. This other column corresponds to the parameter or log file that is determining the maximum number of redo blocks that Oracle processes during recovery. The setting for the parameter in this column is imposing the heaviest requirement on redo block processing.

Tuning the Phases of Instance Recovery

- Besides using checkpoints to tune instance recovery, you can also use a variety of parameters to control Oracle's behavior during the rolling forward and rolling back

phases of instance recovery. In some cases, you can parallelize operations and thereby increase recovery efficiency.

Tuning the Rolling Forward Phase

Use parallel block recovery to tune the roll forward phase of recovery. Parallel block recovery uses a "division of labor" approach to allocate different processes to different data blocks during the roll forward phase of recovery. For example, if the redo log contains a substantial number of entries, process 1 takes responsibility for one part of the log file, process 2 takes responsibility for another part, process 3 takes responsibility for a third part, and so on. Crash, instance, and media recovery of many datafiles on different disk drives are good candidates for parallel block recovery.

Use the RECOVERY_PARALLELISM initialization parameter to specify the number of concurrent recovery processes for instance or media recovery operations. Because crash recovery occurs at instance startup, this parameter is useful for specifying the number of processes to use for crash recovery. The value of this parameter is also the default number of processes used for media recovery if you do not specify the PARALLEL clause of the RECOVER command. The value of this parameter must be greater than 1 and cannot exceed the value of the PARALLEL_MAX_SERVERS parameter. Parallel block recovery requires a minimum of eight recovery processes for it to be more effective than serial recovery.

Recovery is usually I/O bound on reads to data blocks. Consequently, parallelism at the block level may only help recovery performance if it increases total I/Os. In other words, parallelism at the block level by-passes operating system restrictions on asynchronous I/Os. Performance on systems with efficient asynchronous I/O typically does not improve significantly with parallel block recovery.

Tuning the Rolling Back Phase

During the second phase of instance recovery, Oracle rolls back uncommitted transactions. Oracle uses two features, Fast-start on-demand rollback and Fast-start parallel rollback, to increase the efficiency of this recovery phase.

Using Fast-start On-demand Rollback

Using the Fast-start on-demand rollback feature, Oracle automatically allows new transactions to begin immediately after the roll forward phase of recovery completes. Should a user attempt to access a row that is locked by a dead transaction, Oracle rolls back only those changes necessary to complete the transaction, in other words, it rolls them back "on demand." Consequently, new transactions do not have to wait until all parts of a long transaction are rolled back.

Using Fast-start Parallel Rollback

In Fast-start parallel rollback, the background process SMON acts as a coordinator and rolls back a set of transactions in parallel using multiple server processes. Essentially, Fast-start parallel rollback is to rolling back what parallel block recovery is to rolling forward.

Fast-start parallel rollback is mainly useful when a system has transactions that run a long time before committing, especially parallel INSERT, UPDATE, and DELETE operations. When SMON discovers that the amount of recovery work is above a certain threshold, it automatically begins parallel rollback by dispersing the work among several parallel processes: process 1 rolls back one transaction, process 2 rolls back a second transaction, and so on. The threshold is the point at which parallel recovery becomes cost-effective, in other words, when parallel recovery takes less time than serial recovery.

One special form of Fast-start parallel rollback is intra-transaction recovery. In intra-transaction recovery, a single transaction is divided among several processes. For example, assume 8 transactions require recovery with one parallel process assigned to each transaction. The transactions are all similar in size except for transaction 5, which is quite large. This means it takes longer for one process to roll this transaction back than for the other processes to roll back their transactions.

In this situation, Oracle automatically begins intra-transaction recovery by dispersing transaction 5 among the processes: process 1 takes one part, process 2 takes another part, and so on.

You control the number of processes involved in transaction recovery by setting the parameter `FAST_START_PARALLEL_ROLLBACK` to one of three values:

FALSE	Turns off Fast-start parallel rollback.
LOW	Specifies that the number of recovery servers may not exceed twice the value of the CPU_COUNT parameter.
HIGH	Specifies that the number of recovery servers may not exceed four times the value of the CPU_COUNT parameter.

Transparent Application Failover

- Transparent application failover (TAF) is the ability of applications to automatically reconnect to the database if the connection fails. If the client is not involved in a database transaction, then users may not notice the failure of the server.
- During normal client-server database operations, the client maintains a connection to the database so the client and server can communicate. If the server fails, the connection also fails. The next time the client tries to use the connection to execute a new SQL statement, for example, the operating system displays an error to the client. Oracle most commonly then issues the error "ORA-3113: end-of-file on communication channel". At this point, the user must log in to the database again.
- With TAF, however, Oracle automatically obtains a new connection to the database. This allows the user to continue to work using the new connection as if the original connection had never failed.

The MTTR Advisor

- The Mean Time to Recover (MTTR) Advisor in Oracle is a feature introduced to help database administrators optimize recovery performance by providing recommendations for tuning parameters related to instance recovery
- MTTR Advisor helps to increase the instance availability in case of an instance crash or failure. To do this, FAST_START_MTTR_TARGET is set to a non-zero value. FAST_START_MTTR_TARGET is the target for the expected mean time to recovery (MTTR). The result can be found in V\$MTTR_TARGET_ADVICE and V\$INSTANCE_RECOVERY.
- Using MTTR Advisor in Oracle Database, database administrators can optimize recovery performance, minimize downtime, and ensure efficient recovery of their Oracle databases in the event of failures or crashes.
- The FAST_START_MTTR_TARGET initialization parameter is a crucial element in tuning instance recovery times within Oracle. It allows you to specify the desired

- Mean Time To Repair (MTTR), which essentially represents the target time it should take for your database to recover after a crash or instance failure.
- The minimum value for FAST_START_MTTR_TARGET is typically 1 second (though Oracle might round it up). However, achieving a very low MTTR might not be practical due to limitations in hardware performance.
- The maximum value is 3600 seconds (1 hour). Setting a very high target essentially disables the fast-start checkpointing mechanism.
- The following example shows how to set the value of FAST_START_MTTR_TARGET:

```
SQL> ALTER SYSTEM SET FAST_START_MTTR_TARGET=30;
```

Tuning FAST_START_MTTR_TARGET and Using MTTR Advisor

- To determine the appropriate value for FAST_START_MTTR_TARGET for your database, use the following four step process:
 1. *Calibrate the FAST_START_MTTR_TARGET*
 2. *Determine the Practical Range for FAST_START_MTTR_TARGET*
 3. *Evaluate Different Target Values with MTTR Advisor*
 4. *Determine the Optimal Size for Redo Logs*

Calibrate the FAST_START_MTTR_TARGET

- The FAST_START_MTTR_TARGET initialization parameter causes the database to calculate internal system trigger values, in order to limit the length of the redo log and the number of dirty data buffers in the data cache. This calculation uses estimated time to read a redo block, estimates of the time to read and write a data block and characteristics of typical workload of the system, such as how many dirty buffers corresponds to how many change vectors, and so on.
- Initially, internal defaults are used in the calculation. These defaults are replaced over time by data gathered on I/O performance during system operation and actual cache recoveries.
- You will have to perform several instance recoveries in order to calibrate your FAST_START_MTTR_TARGET value properly. Before starting calibration, you must decide whether FAST_START_MTTR_TARGET is being calibrated for a database crash or a hardware crash. This is a consideration if your database files

are stored in a file system or if your I/O subsystem has a memory cache, because there is a considerable difference in the read and write time to disk depending on whether the files are cached. The appropriate value for FAST_START_MTTR_TARGET will depend upon which type of crash is more important to recover from quickly.

- To effectively calibrate FAST_START_MTTR_TARGET, ensure that you run the typical workload of the system for long enough, and perform several instance recoveries to ensure that the time to read a redo block and the time to read or write a data block during recovery are recorded accurately.

Determine the Practical Range for FAST START MTTR TARGET

- After calibration, you can perform tests to determine the practical range for FAST_START_MTTR_TARGET for your database.

Determining Lower Bound for FAST_START_MTTR_TARGET: Scenario

- To determine the lower bound of the practical range, set FAST_START_MTTR_TARGET to 1, and start up your database. Then check the value of V\$INSTANCE_RECOVERY.TARGET_MTTR, and use this value as a good lower bound for FAST_START_MTTR_TARGET. Database startup time, rather than cache recovery time, is usually the dominant factor in determining this limit.
- For example, set the FAST_START_MTTR_TARGET to 1:

```
SQL> ALTER SYSTEM SET FAST_START_MTTR_TARGET=1;
```

- Then, execute the following query immediately after opening the database:

```
SQL> SELECT TARGET_MTTR, ESTIMATED_MTTR
        FROM V$INSTANCE_RECOVERY;
```

- Oracle Database responds with the following:

```
TARGET_MTTR ESTIMATED_MTTR
18          15
```

The TARGET_MTTR value of 18 seconds is the minimum MTTR target that the system can achieve, that is, the lowest practical value for FAST_START_MTTR_TARGET. This minimum is calculated based on the average database startup time.

- The ESTIMATED_MTTR field contains the estimated mean time to recovery based on the current state of the running database. Because the database has just opened, the system contains few dirty buffers, so not much cache recovery would be required if the instance failed at this moment. That is why ESTIMATED_MTTR can, for the moment, be lower than the minimum possible TARGET_MTTR.
- ESTIMATED_MTTR can be affected in the short term by recent database activity. Assume that you query V\$INSTANCE_RECOVERY immediately after a period of heavy update activity in the database. Oracle Database responds with the following:

```
TARGET_MTTR ESTIMATED_MTTR
18          30
```

Now the effective MTTR target is still 18 seconds, and the estimated MTTR (if a crash happened at that moment) is 30 seconds. This is an acceptable result. This means that some checkpoints writes might not have finished yet, so the buffer cache contains more dirty buffers than targeted.

- Now wait for sixty seconds and reissue the query to V\$INSTANCE_RECOVERY. Oracle Database responds with the following:

```
TARGET_MTTR ESTIMATED_MTTR
18          25
```

The estimated MTTR at this time has dropped to 25 seconds, because some of the dirty buffers have been written out during this period

Determining Upper Bound for FAST_START_MTTR_TARGET

- To determine the upper bound of the practical range, set FAST_START_MTTR_TARGET to 3600, and operate your database under a

typical workload for a while. Then check the value of `V$INSTANCE_RECOVERY.TARGET_MTTR`. This value is a good upper bound for `FAST_START_MTTR_TARGET`.

The procedure is substantially similar to that in "Determining Lower Bound for `FAST_START_MTTR_TARGET`: Scenario".

Selecting Preliminary Value for FAST_START_MTTR_TARGET

- After you have determined the practical bounds for the `FAST_START_MTTR_TARGET` parameter, select a preliminary value for the parameter. Choose a higher value within the practical range if your concern is with database performance, and a lower value within the practical range if your priority is shorter recovery times. The narrower the practical range, of course, the easier the choice becomes.
- For example, if you discovered that the practical range was between 17 and 19 seconds, it would be quite simple to choose 19, because it makes relatively little difference in recovery time and at the same time minimizes the effect of checkpointing on system performance. However, if you found that the practical range was between 18 and 40 seconds, you might choose a compromise value of 30, and set the parameter accordingly:

```
SQL> ALTER SYSTEM SET FAST_START_MTTR_TARGET=30;
```

You might then go on to use the MTTR Advisor to determine an optimal value.

Evaluate Different Target Values with MTTR Advisor

- After you have selected a preliminary value for `FAST_START_MTTR_TARGET`, you can use MTTR Advisor to evaluate the effect of different `FAST_START_MTTR_TARGET` settings on system performance, compared to your chosen setting.

Enabling MTTR Advisor

- To enable MTTR Advisor, set the two initialization parameters STATISTICS_LEVEL and FAST_START_MTTR_TARGET.
- STATISTICS_LEVEL governs whether all advisors are enabled and is not specific to MTTR Advisor. Ensure that it is set to TYPICAL or ALL. Then, when FAST_START_MTTR_TARGET is set to a nonzero value, the MTTR Advisor is enabled.

Using MTTR Advisor

- After enabling MTTR Advisor, run a typical database workload for a while. When MTTR Advisor is ON, the database simulates checkpoint queue behavior under the current value of FAST_START_MTTR_TARGET, and up to four other different MTTR settings within the range of valid FAST_START_MTTR_TARGET values. (The database will in this case determine the valid range for FAST_START_MTTR_TARGET itself before testing different values in the range.)

Viewing MTTR Advisor Results: V\$MTTR_TARGET_ADVICE

- The dynamic performance view V\$MTTR_TARGET_ADVICE lets you view statistics or advisories collected by MTTR Advisor.
- The database populates V\$MTTR_TARGET_ADVICE with advice about the effects of each of the FAST_START_MTTR_TARGET settings for your database. For each possible value of FAST_START_MTTR_TARGET, the row contains details about how many cache writes would be performed under the workload tested for that value of FAST_START_MTTR_TARGET.
- Specifically, each row contains information about cache writes, total physical writes (including direct writes), and total I/O (including reads) for that value of FAST_START_MTTR_TARGET, expressed both as a total number of operations and a ratio compared to the operations under your chosen FAST_START_MTTR_TARGET value. For instance, a ratio of 1.2 indicates 20% more cache writes.
- Knowing the effect of different FAST_START_MTTR_TARGET settings on cache write activity and other I/O enables you to decide better

which FAST_START_MTTR_TARGET value best fits your recovery and performance needs.

- If MTTR Advisor is currently on, then V\$MTTR_TARGET_ADVICE shows the Advisor information collected. If MTTR Advisor is currently OFF, then the view shows information collected the last time MTTR Advisor was ON since database startup, if any. If the database has been restarted since the last time the MTTR Advisor was used, or if it has never been used, the view will not show any rows.

Determine the Optimal Size for Redo Logs

- You can use the V\$INSTANCE_RECOVERY view column OPTIMAL_LOGFILE_SIZE to determine the size of your online redo logs. This field shows the redo log file size in megabytes that is considered optimal based on the current setting of FAST_START_MTTR_TARGET. If this field consistently shows a value greater than the size of your smallest online log, then you should configure all your online logs to be at least this size.
- Note, however, that the redo log file size affects the MTTR. In some cases, you may be able to refine your choice of the optimal FAST_START_MTTR_TARGET value by re-running the MTTR Advisor with your suggested optimal log file size.

Media Failure

An error can occur when trying to write or read a file on disk that is required to operate an Oracle database. This occurrence is called **media failure** because there is a physical problem reading or writing to files on the storage medium.

A common example of media failure is a disk head crash that causes the loss of all database files on a disk drive. All files associated with a database are vulnerable to a disk crash, including datafiles, control files, online redo logs, and archived logs.

The appropriate recovery from a media failure depends on the files affected. Media failure is the primary concern of a backup and recovery strategy, because it typically requires restoring some or all database files and the application of redo during recovery.

How Media Failures Affect Database Operation

Media failures can affect one or all types of files necessary for the operation of an Oracle database, including datafiles, online redo log files, and control files. Also, media failures can affect archived redo logs stored on disk.

Database operation after a media failure of online redo log files or control files depends on whether the online redo log or control file is protected by multiplexing, as recommended. When an online redo log or control file is multiplexed, multiple copies of the file are maintained on the system. Typically, multiplexed files are stored on separate disks.

If a media failure damages a single disk, and if you have a multiplexed online redo log, then the database can usually continue to operate without significant interruption. Damage to a nonmultiplexed online redo log causes database operation to halt and may cause permanent loss of data. Damage to any control file, whether it is multiplexed or not, halts database operation once Oracle attempts to read or write to the damaged control file, which happens frequently, for example at every checkpoint and log switch.

Media failures that affect datafiles can be divided into two categories: read errors and write errors. In a read error, Oracle discovers it cannot read a datafile and an operating system error is returned to the application, along with an Oracle error indicating that the file cannot be found, cannot be opened, or cannot be read. Oracle continues to run, but the error is returned each time an unsuccessful read occurs. At the next checkpoint, a write error will occur when Oracle attempts to write the file header as part of the standard checkpoint process.

If Oracle discovers that it cannot write to a datafile, and if Oracle is in ARCHIVELOG mode, then Oracle returns an error in the database writer trace file and takes the datafile offline automatically. Only the datafile that cannot be written to is taken offline; the tablespace containing that file remains online.

If the datafile that cannot be written to is in the SYSTEM tablespace, then the file is not taken offline. Instead, an error is returned and Oracle shuts down the instance. The reason for this exception is that all files in the SYSTEM tablespace must be online in order for Oracle to operate properly. For the same reason, the undo tablespaces (if in automatic undo

management mode) or the datafiles of a tablespace containing active rollback segments (if in manual undo management mode) must remain online.

If Oracle cannot write to a datafile, and Oracle is not archiving the filled online redo log files, then the database writer background process fails and the current instance fails. If the problem is temporary (for example, the disk controller was powered off), then crash or instance recovery usually can be performed using the online redo log files, in which case the instance can be restarted. However, if a datafile is permanently damaged and archiving is not used, then you must restore the entire database using the most recent consistent backup.

Configure a Database for Recoverability

Configuring a database for recoverability in Oracle involves implementing a comprehensive set of practices and features to ensure that data can be restored and recovered efficiently in the event of failures or disasters. Here's a step-by-step guide to configuring a database for recoverability in Oracle:

Implement Regular Backups:

- Establish a backup strategy that includes regular backups of the database files, including datafiles, control files, and archived redo logs.
- Determine the appropriate backup frequency based on your organization's recovery point objectives (RPO) and retention policies.
- Use Oracle Recovery Manager (RMAN) to automate backup tasks and manage backup sets.

Utilize Redundancy and High Availability Features:

- Implement redundancy and high availability features such as Oracle Data Guard, Oracle Real Application Clusters (RAC), or Oracle Active Data Guard to ensure database availability and data protection.
- Configure standby databases for disaster recovery purposes and set up Data Guard to automatically synchronize changes between the primary and standby databases.

Enable Flashback Technology:

- Oracle Flashback Technology provides a set of features for quickly recovering the database to a previous point in time without restoring from backups.
- Enable Flashback Database to restore the entire database to a specific point in time, or use Flashback Table to recover individual tables or rows.

Configure Fast Recovery Area (FRA):

- Oracle's Fast Recovery Area provides a centralized location for storing backups, archived redo logs, and other recovery-related files.
- Configure a sufficient FRA size to accommodate backup and recovery operations, and set up automatic space management for the FRA.

Optimize Redo Log Configuration:

- Properly size and configure the redo log files to handle the workload of the database.
- Monitor redo log size and switch frequency to ensure that the redo log files are not too small or too large, which can impact recovery performance.

Monitor and Test Backups:

- Regularly monitor backup jobs and review backup logs to ensure that backups are completing successfully.
- Perform periodic backup validation and testing to verify the integrity of backups and the effectiveness of recovery procedures.

Document and Maintain Recovery Procedures:

- Document recovery procedures, including steps for restoring backups, recovering the database, and activating standby databases.
- Keep recovery documentation up to date and ensure that relevant personnel are familiar with recovery procedures through training and drills.
- Regularly Review and Update Configuration:
- Periodically review and update database configuration settings and recovery procedures based on changes in workload, technology advancements, and best practices.

- Stay informed about new features and capabilities introduced in Oracle Database releases that can enhance recoverability and data protection.

By following these steps and best practices, you can configure your Oracle database for recoverability, ensuring that your data remains protected and available in the event of failures or disasters.

Hot backup vs Cold Backup in Oracle

- In Oracle database management, "hot backup" and "cold backup" refer to two different methods of backing up a database, each with its own advantages and considerations.

Hot Backup:

- A hot backup, also known as an online backup, is taken while the database is still running and actively processing transactions.
 - During a hot backup, Oracle's built-in features, such as Recovery Manager (RMAN) or third-party backup tools, are used to capture database files, including datafiles, control files, and archived redo logs, while the database remains operational.
 - Hot backups leverage Oracle's ability to create consistent backups even while the database is in use by managing the backup process in a way that ensures data integrity and recoverability.
- ☞ You can use Oracle RMAN to initiate a hot backup while the database is online and processing transactions. RMAN will leverage features like block change tracking and incremental backups to capture only the changed blocks since the last backup, minimizing backup time and resource usage. Users can continue to access the database during the backup process, ensuring continuous operation of critical applications

Advantages:

- Minimizes downtime as the database remains accessible to users during the backup process.
- Allows for continuous operation of critical applications without interruption.

Considerations:

- Requires careful management and coordination to ensure data consistency and avoid performance degradation during backup operations.
- May introduce additional overhead on the system due to increased I/O activity during backup.

Cold Backup:

- A cold backup, also known as an offline backup, is taken while the database is shut down and not actively processing transactions.
- Before initiating a cold backup, the Oracle database instance is cleanly shut down to ensure that all transactions are committed, and database files are in a consistent state.
- Once the database is offline, backup tools or manual file copy operations are used to capture database files, including datafiles, control files, and archived redo logs.
- After the backup is completed, the database instance can be restarted and made available to users.

☞ In a cold backup, you would first shut down the Oracle database instance cleanly to ensure that all transactions are committed and the database is in a consistent state. Once the database is offline, you can use backup tools or manual file copy operations to capture the database files. After completing the backup, you would restart the database instance to make it available to users again. During the backup process, users would not be able to access the database, resulting in downtime for business operations. However, a cold backup ensures simplicity and reliability since there is no need to manage backup operations while the database is running

Advantages:

- Simplicity and reliability, as there is no need to manage backup operations while the database is actively processing transactions.
- Avoids the potential for data inconsistencies or performance issues during backup operations..

Considerations:

- Requires downtime for the database, which may impact business operations and user accessibility.
- Recovery time may be longer compared to hot backups since the database must be brought online and possibly undergo instance recovery after restoration.